Understand ClassroomCast: requirements, workflows and activity diagrams

Prepared 2 November 2026 from the two project descriptions supplied in the latest request. They contain the same project requirements, with different formatting. This guide explains those requirements before team/Scrum assignment. It does not change the supplied priorities or claim any flow is implemented.

1. What are we building?

ClassroomCast lets a teacher broadcast a screen and supported audio to students' browsers on a local network, while authorized administrators manage classroom network restrictions through software.

Imagine a programming lesson: the teacher opens a class, displays its join code, and students join on classroom Wi-Fi. The teacher shares an IDE screen. An administrator-approved Programming Lab profile restricts selected AI domains while allowing approved documentation. When the class ends, students disconnect and the previous network policy is restored.

There are four product areas:

Area	User need	Visible result
Classroom broadcasting	See/hear the teacher's selected content	Live screen/audio playback
Classroom management	Control the session and participation	Join code, admission, roster, remove/block, end class
Network access control	Manage supported restrictions from a dashboard	Profiles, exceptions, actual enforcement state, restoration
Deployment and diagnostics	Operate on campus or portable local infrastructure	Reachable local services, health checks, configuration and recovery

Local network is different from internet. Laptops can exchange classroom data through a router/access point without an internet connection. With no internet, local broadcasting still works, but public websites may be unavailable regardless of filtering. To demonstrate selective public-domain restrictions, test on a controlled network with upstream connectivity; do not treat WAN disconnection as proof that domain blocking works.

2. Actors and system responsibilities

Actor/component	Responsibility
Teacher	Create/run/end classes, select capture source, manage participation
Student	Join and view/listen; use permitted participation/material features

Actor/component	Responsibility
ClassroomCast administrator	Configure policy profiles, exceptions, expiry and emergency restoration
Campus IT / authorized installer	Approve and provision hosting, names/certificates, network reachability and enforcement integration
Backend	Validate access; manage sessions, participants and policy lifecycle; record outcomes
SFU media server	Receive the teacher's media and forward it to viewers
Enforcement service	Apply supported domain/network rules to traffic routed through its scope

A teacher and administrator may be the same person only when that person has both permissions. A normal student browser has no authority to change other laptops' network access.

3. Requirements versus choices

Must means required for the described release. **Should** means important but lower priority. **Could** means optional. Campus and router requirements are listed without separate MoSCoW labels; they are deployment obligations in the supplied specification, not automatically optional.

The source recommends technology options. React/Vite, NestJS, LiveKit, SQLite and AdGuard Home are proposed choices from our earlier plan, not the only permissible implementation. The required architecture is local services with an SFU, secure capture context, server-side secrets, diagnostics/recovery, and distinct configured/actual enforcement state.

The source does **not** define exact latency, expiry timing, certificate distribution, student identity strength, join-code lifetime, mandatory approval on every join, or behavior when enforcement fails. Any such detail below is labeled a proposed decision. Earlier planning numbers such as 1-second latency are proposed acceptance targets, not requirements copied from this source.

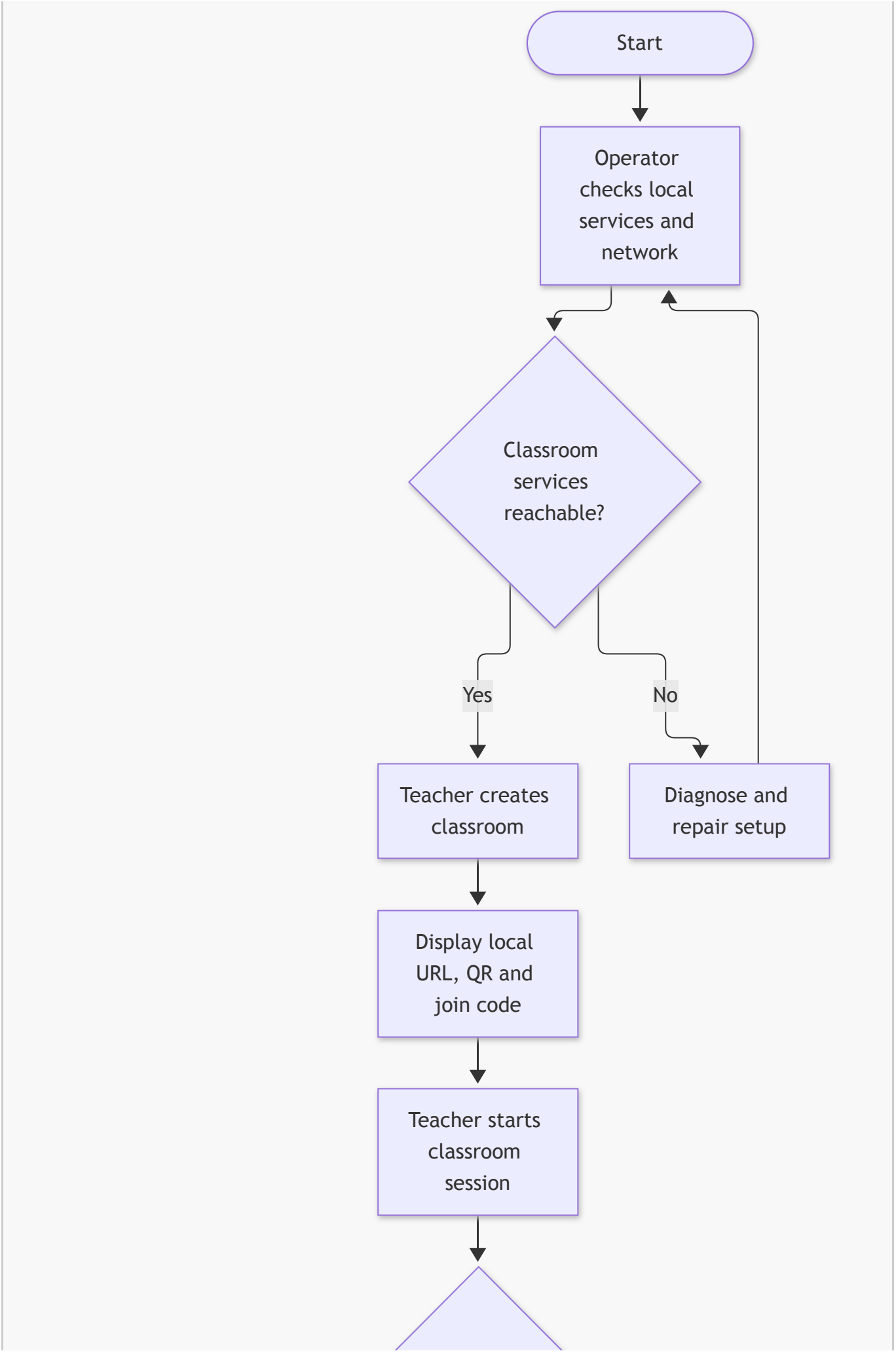
4. Reading the diagrams

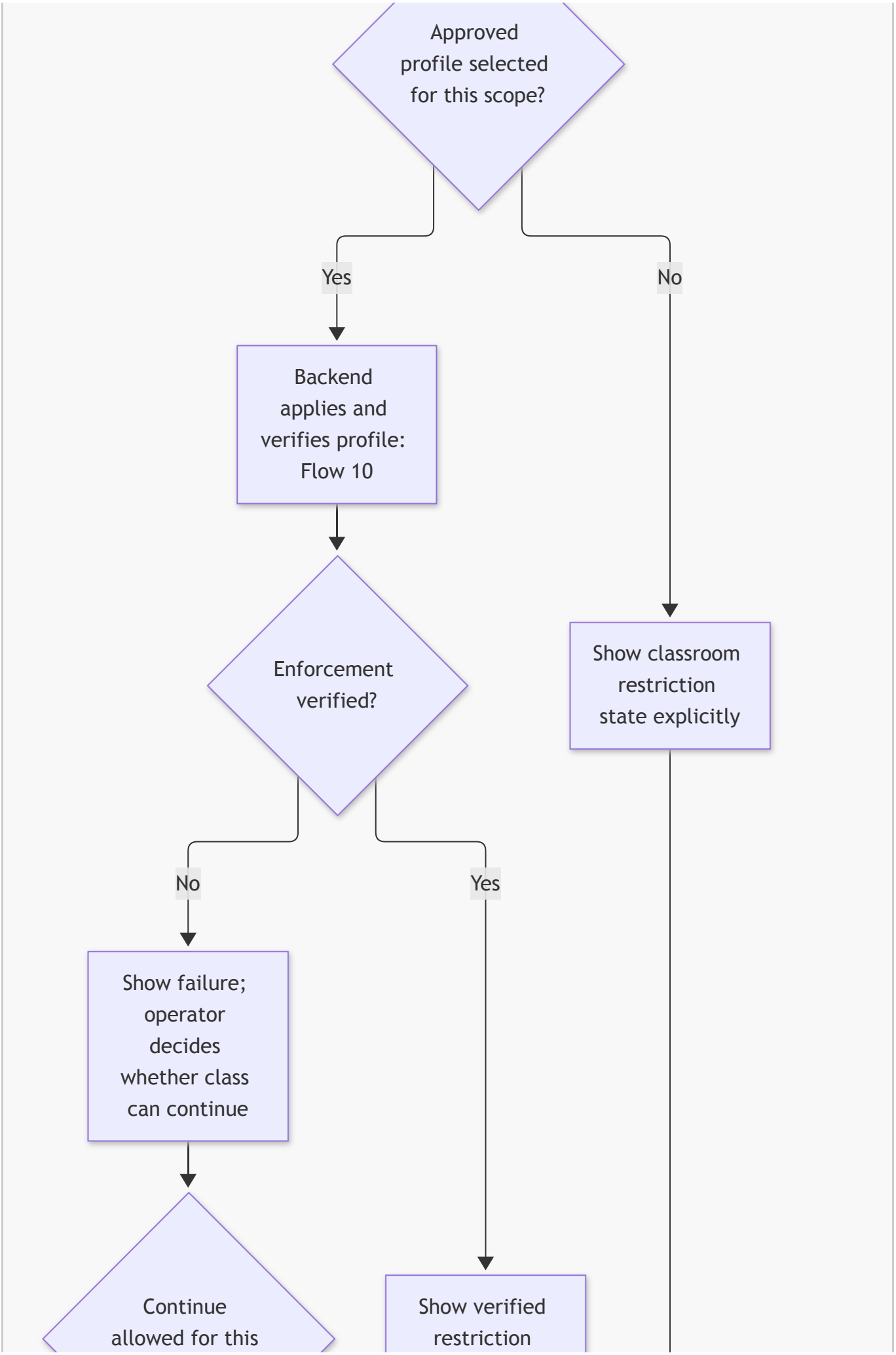
These are simplified activity diagrams expressed in Mermaid: rounded Start/End nodes, action boxes, decision diamonds and labeled paths. Actor names in action labels identify responsibility. Each diagram describes a process rather than a screen layout or software class structure.

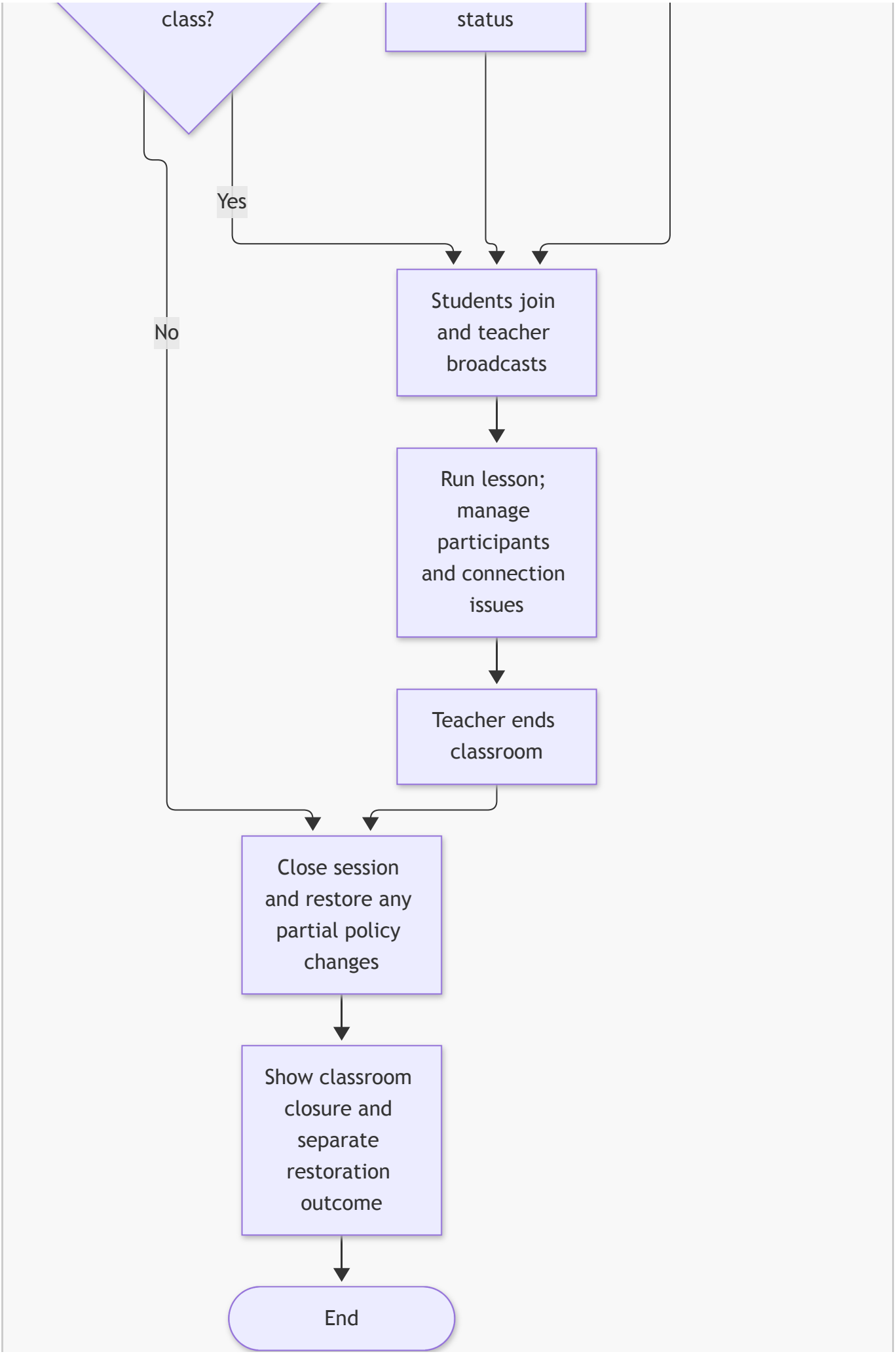
Most diagrams show one invocation. Long-running classroom activity repeats and can have multiple concurrent students. Flow 01 establishes the order; flows 02–18 explain the branches in detail.

Flow 01 — Entire classroom lifecycle

Trigger: A prepared classroom is about to begin. **Outcome:** Session closed, participants disconnected, policy restoration completed or visibly pending/failed. Starting a class and starting capture are separate actions.


Start
Operator checks local services and network
Classroom services reachable?
Yes
No
Teacher creates classroom
Diagnose and repair setup
Display local URL, QR and join code
Teacher starts classroom session

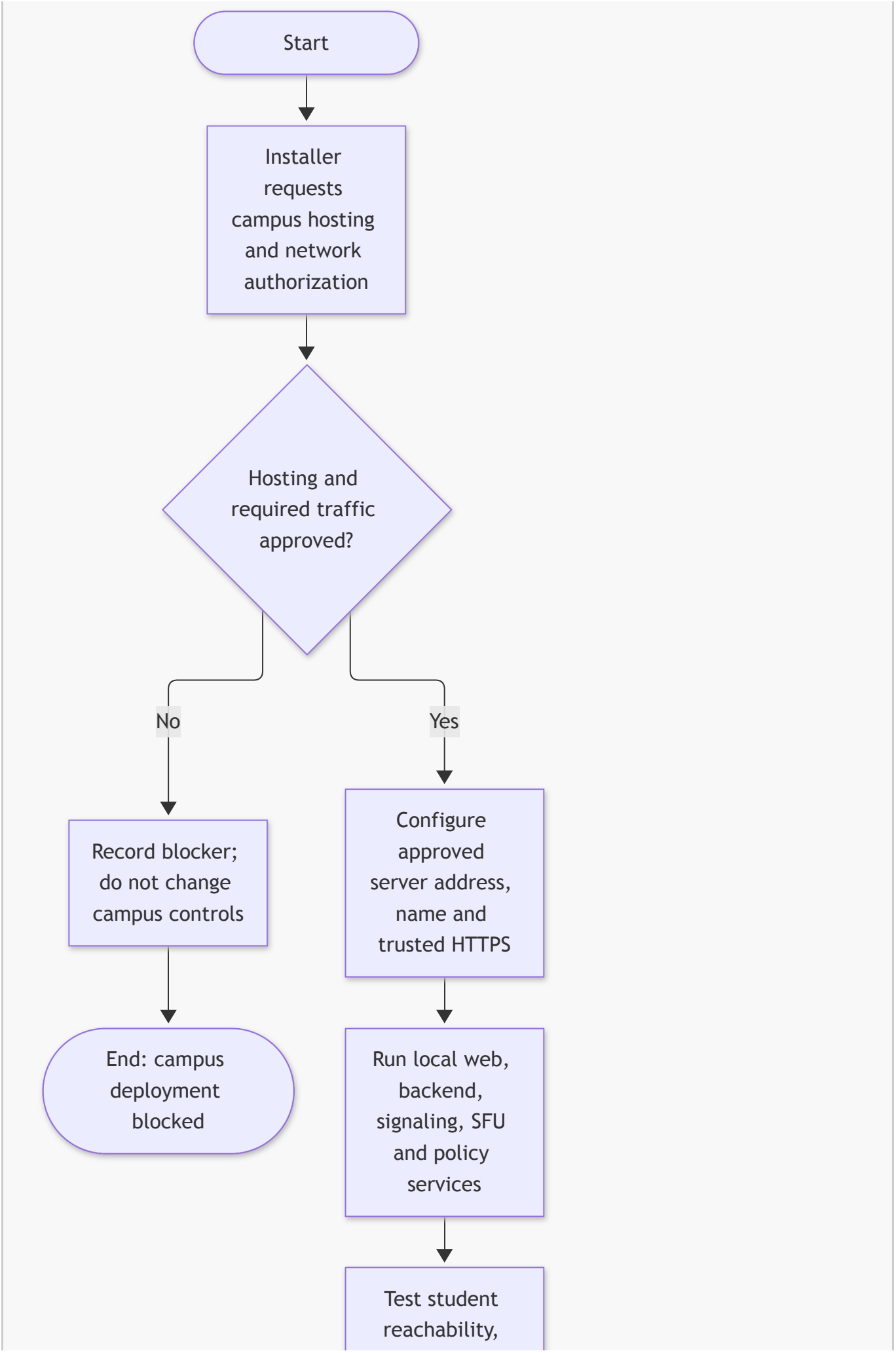

Approved profile selected for this scope?
Yes
No
Backend applies and verifies profile: Flow 10
Enforcement verified?
Show classroom restriction state explicitly
No
Yes
Show failure; operator decides whether class can continue
Continue allowed for this
Show verified restriction

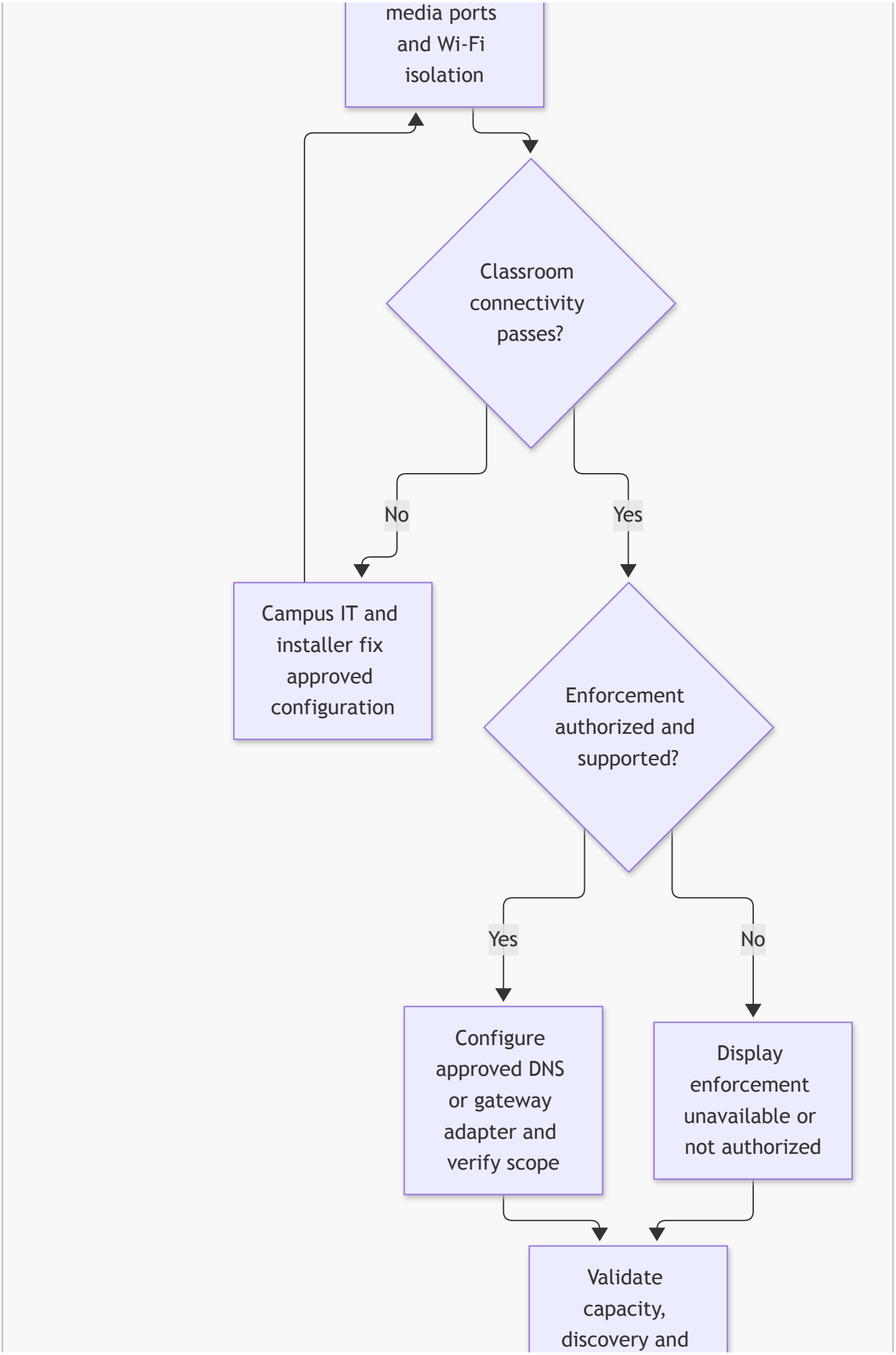

class?
status
Yes
No
Students join and teacher broadcasts
Run lesson; manage participants and connection issues
Teacher ends classroom
Close session and restore any partial policy changes
Show classroom closure and separate restoration outcome
End

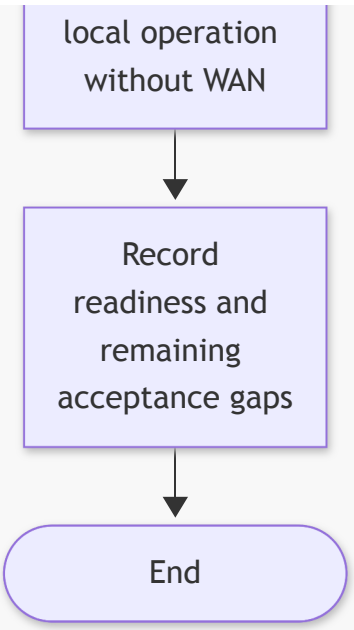
The decision to continue without verified restrictions is unresolved. Proposed behavior: display the problem and allow an authorized operator to decide; an exam may require stopping, while a normal lesson may continue. Never display a failed policy as active.

Flow 02 — Prepare campus deployment

Covers CN01–CN10. This is installation/network preparation, generally performed before classes. Teachers should not need to edit router rules for every lesson.


Start
Installer requests campus hosting and network authorization
Hosting and required traffic approved?
No
Yes
Record blocker; do not change campus controls
End: campus deployment blocked
Configure approved server address, name and trusted HTTPS
Run local web, backend, signaling, SFU and policy services
Test student reachability,

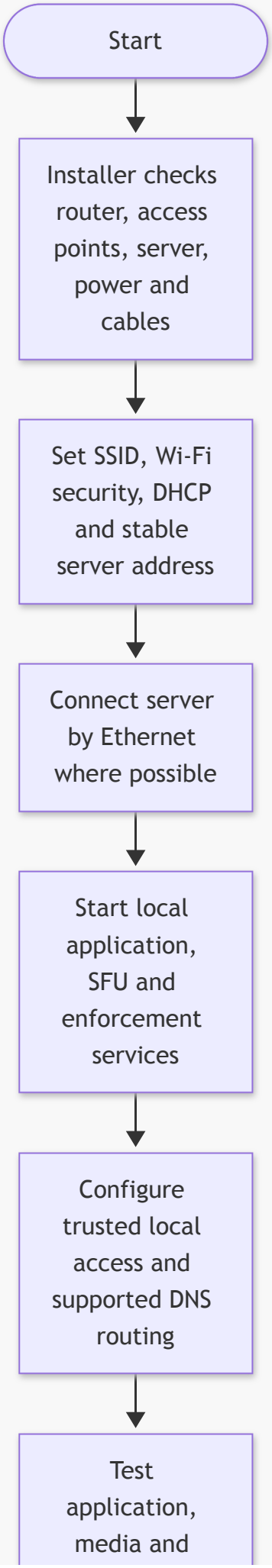

media ports and Wi-Fi isolation
Classroom connectivity passes?
No
Yes
Campus IT and installer fix approved configuration
Enforcement authorized and supported?
Yes
No
Configure approved DNS or gateway adapter and verify scope
Display enforcement unavailable or not authorized
Validate capacity, discovery and

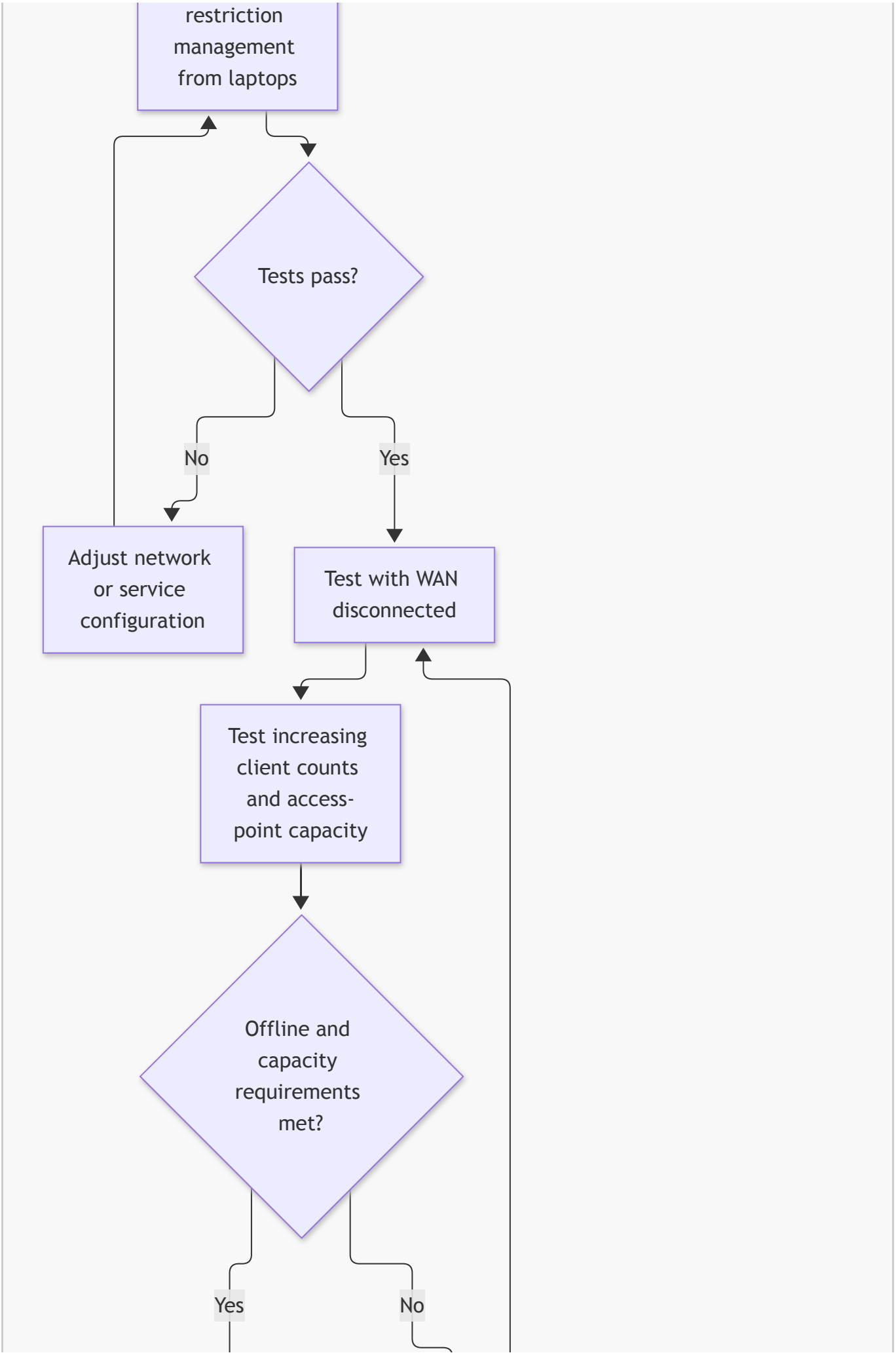


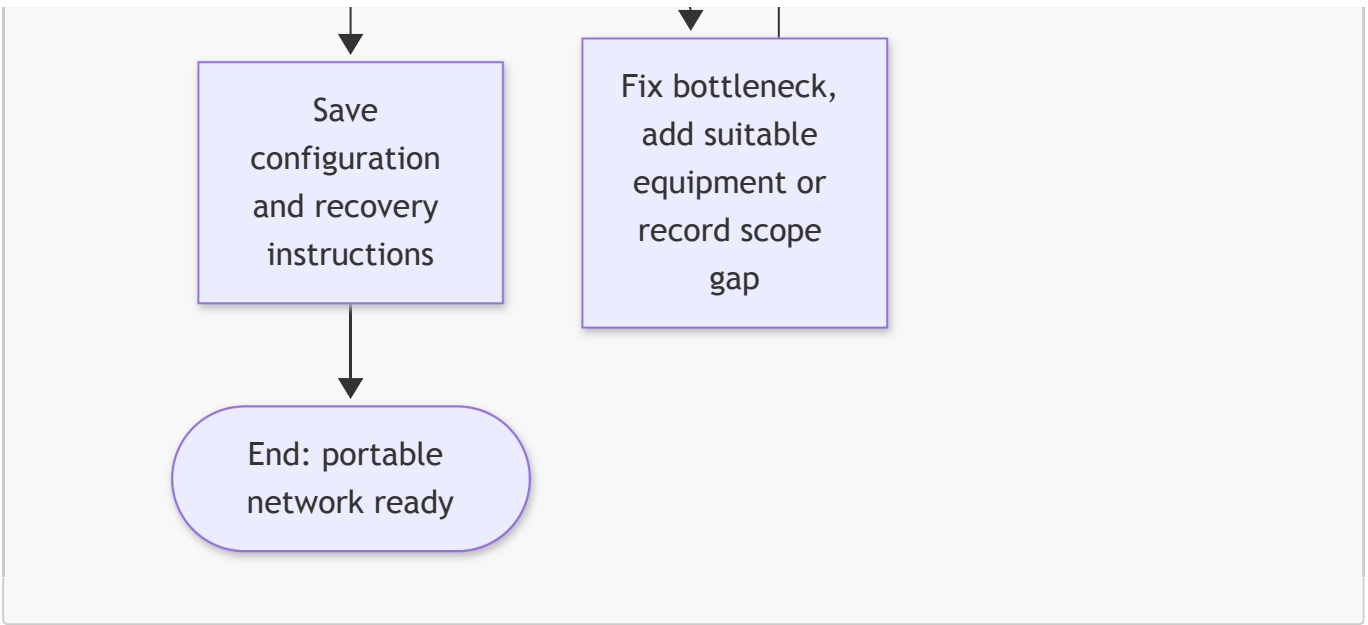
Campus browsing restrictions remain an acceptance gap if no approved integration exists. The system can expose that limitation accurately while classroom connectivity is tested; this does not mean CN10 is satisfied.

Flow 03 — Prepare portable deployment

Covers OR01–OR12. A teacher laptop may host the services if suitable; a separate local mini-PC/server is another option. Browser-only students do not mean server-free deployment.


Start
Installer checks router, access points, server, power and cables
Set SSID, Wi-Fi security, DHCP and stable server address
Connect server by Ethernet where possible
Start local application, SFU and enforcement services
Configure trusted local access and supported DNS routing
Test application, media and

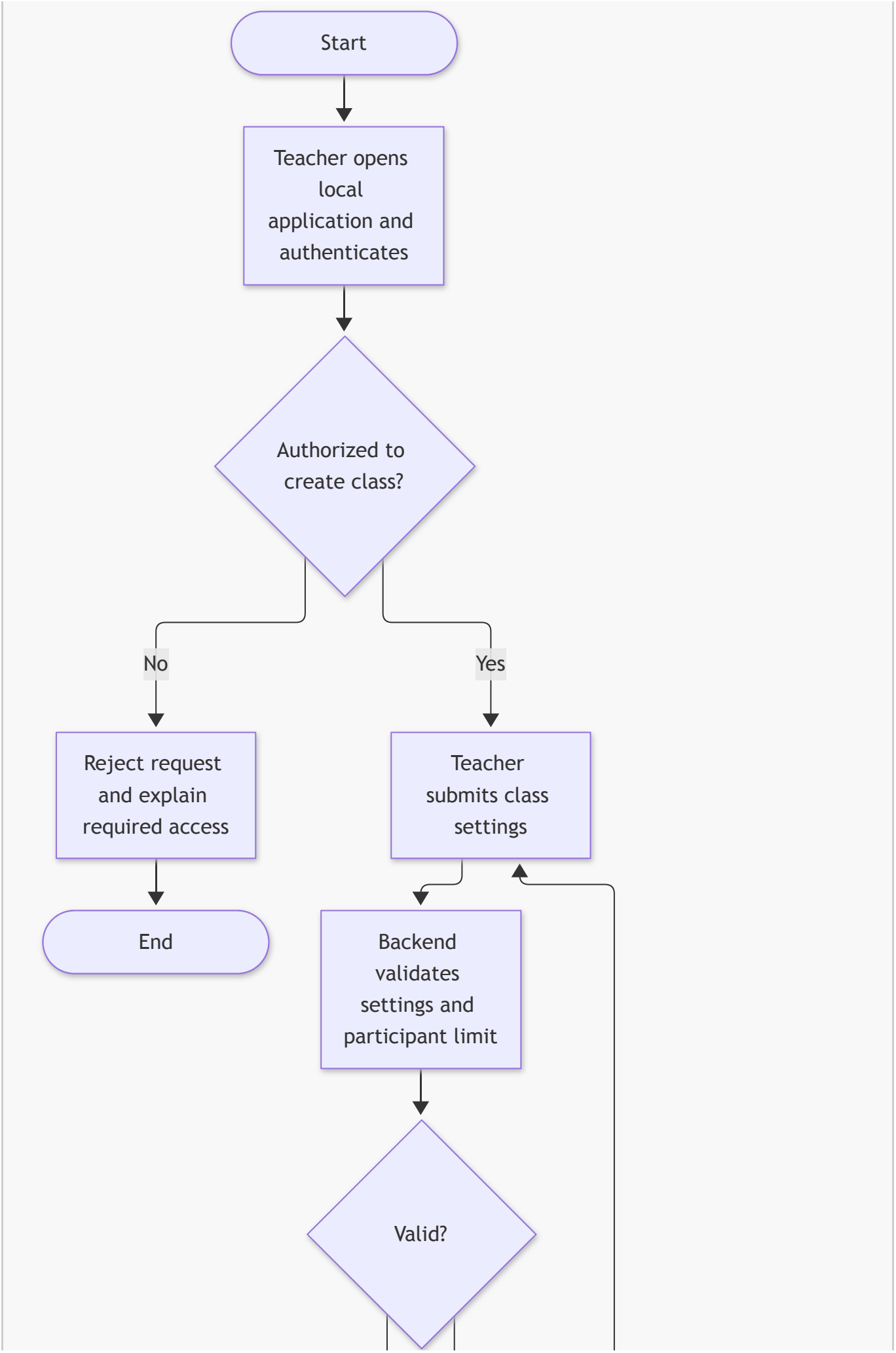

restriction management from laptops
Tests pass?
No
Yes
Adjust network or service configuration
Test with WAN disconnected
Test increasing client counts and access-point capacity
Offline and capacity requirements met?
Yes
No

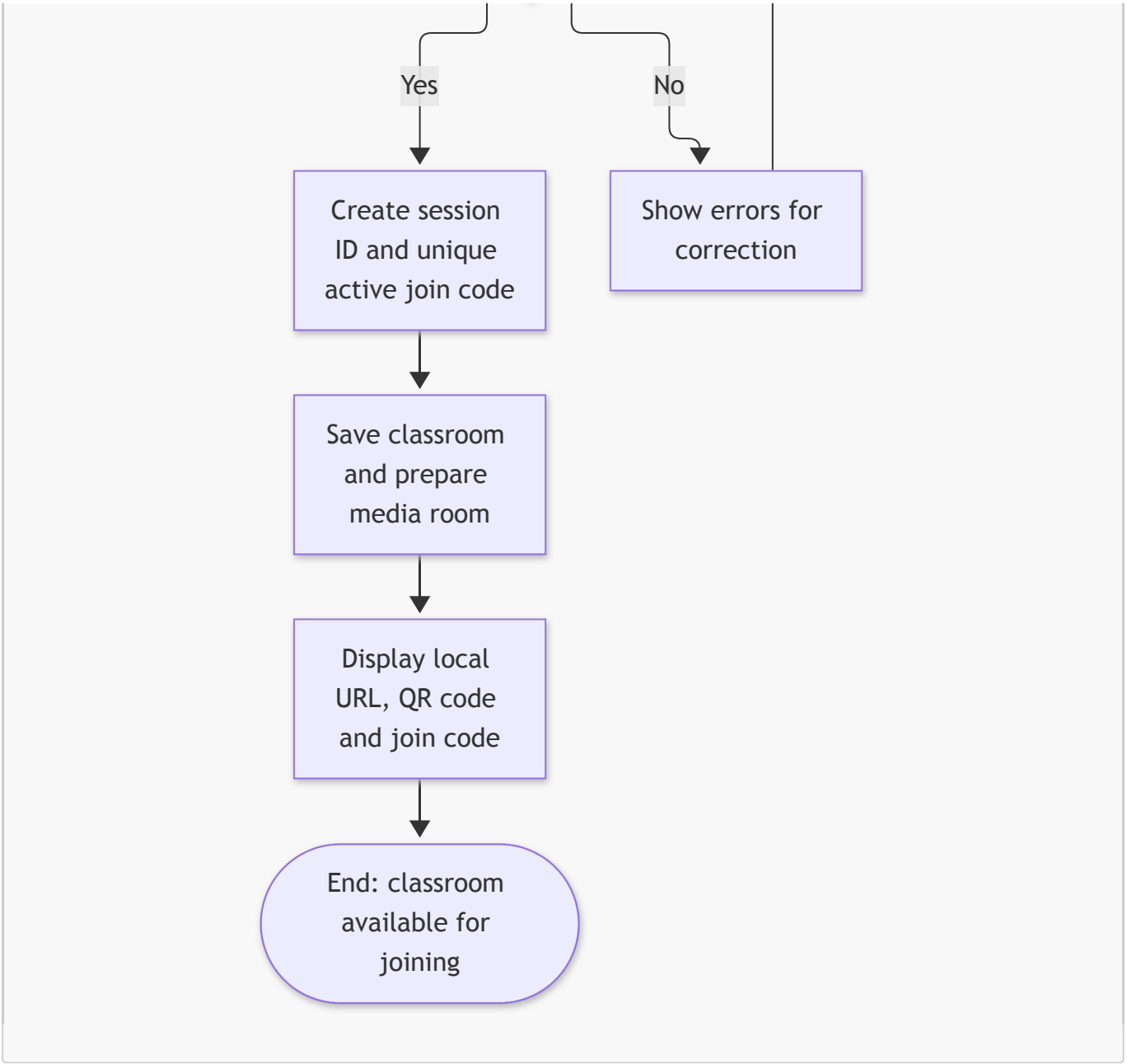


The example IP addresses in the source are examples, not fixed requirements. One router is not automatically sufficient for 150 active viewers; capacity and access-point needs must be measured.

Flow 04 — Teacher creates a classroom

Covers T01, T08 and classroom administration. Authentication is needed to enforce teacher authority; the source does not prescribe a particular login provider or token format.

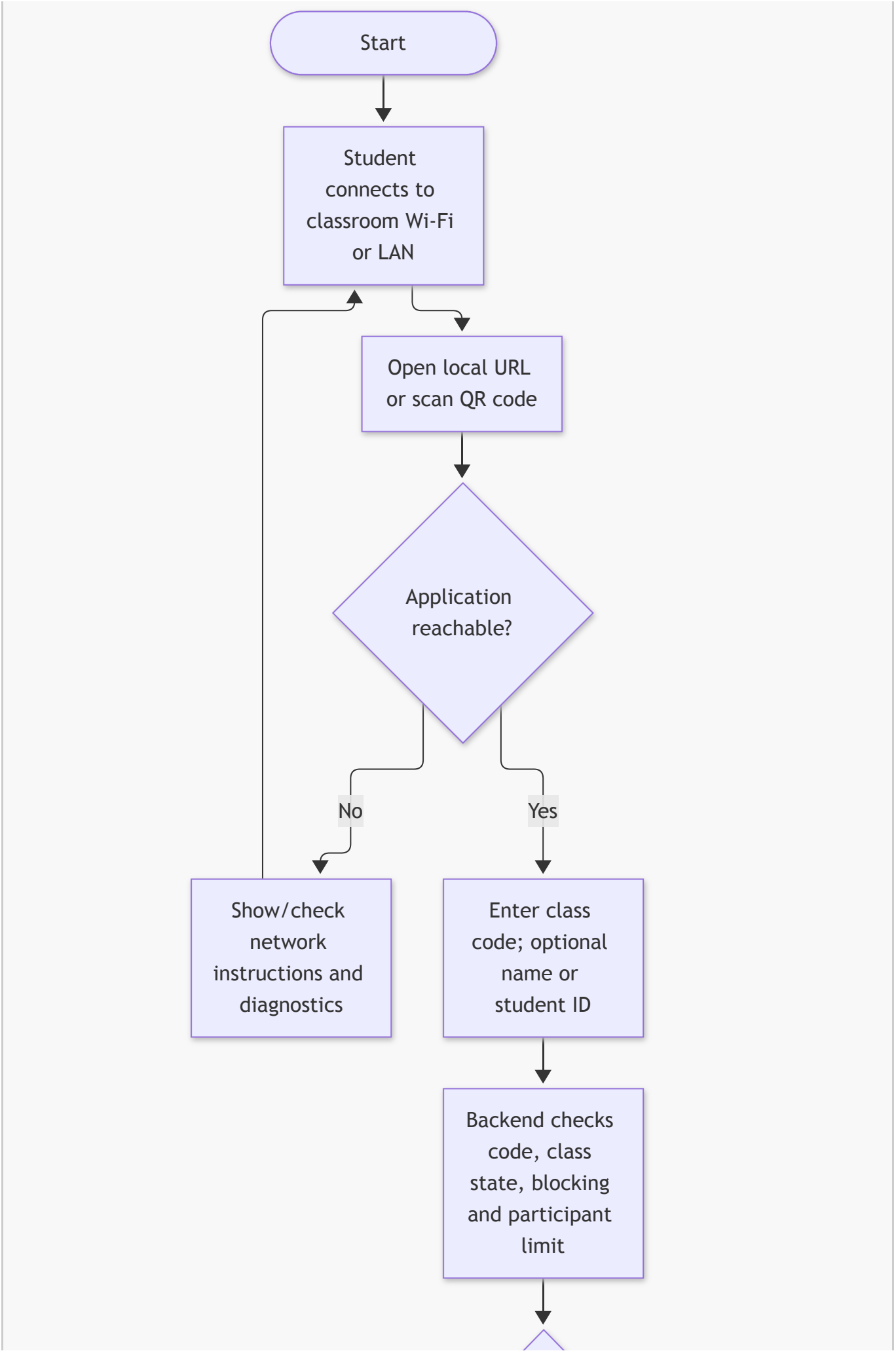

Start
Teacher opens local application and authenticates
Authorized to create class?
No
Yes
Reject request and explain required access
End
Teacher submits class settings
Backend validates settings and participant limit
Valid?

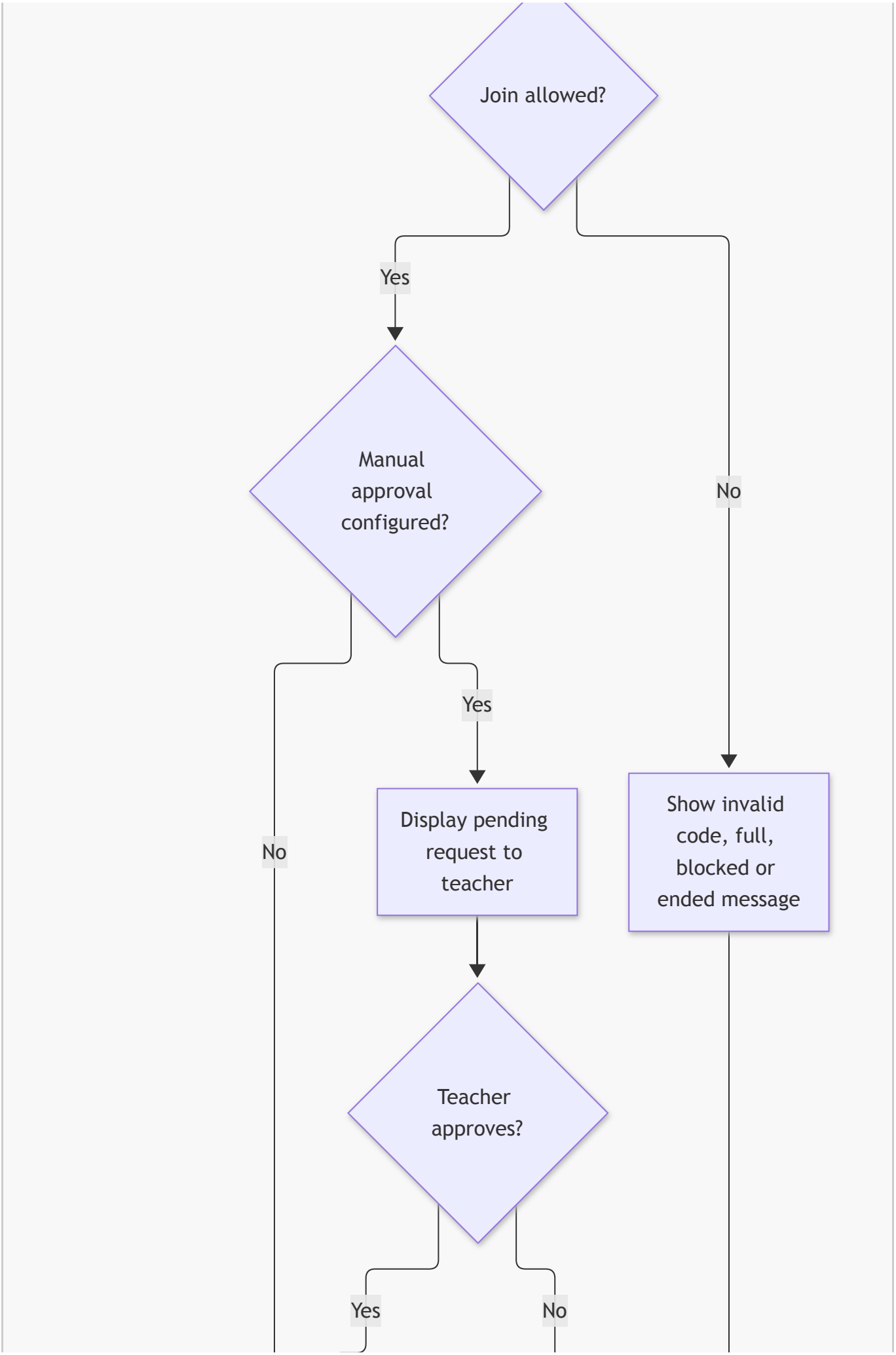


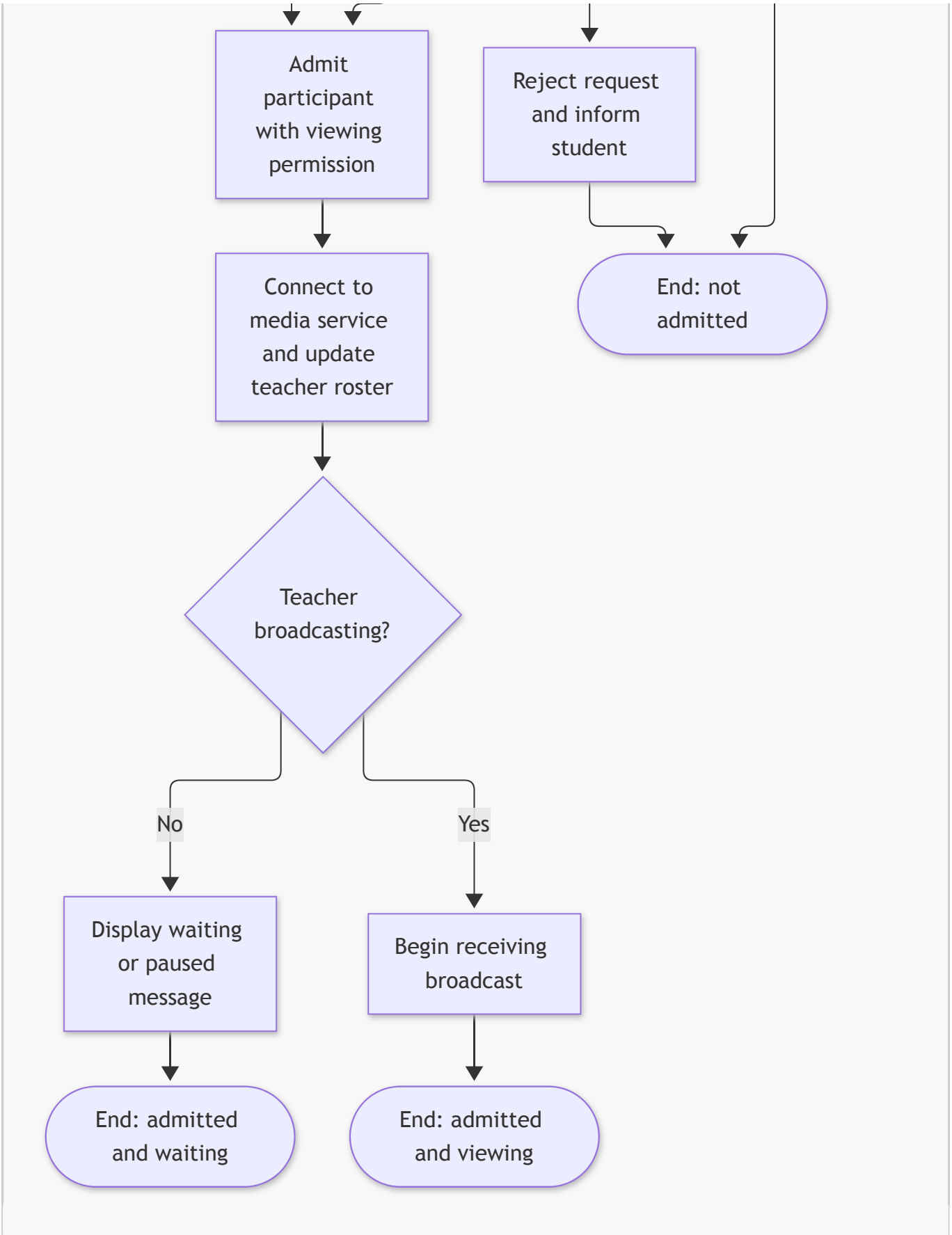
URL locates the application; **QR code** is a convenient way to open joining information; **join code** identifies/admit access to the session. A QR code does not create network connectivity by itself.

Flow 05 — Student joins and is admitted

Covers S01–S02, S07, T05–T06; optional S08. Mandatory manual approval for every student is a design choice. The system must support teacher approval/rejection, but the default admission mode needs agreement.


Start
Student connects to classroom Wi-Fi or LAN
Open local URL or scan QR code
Application reachable?
No
Yes
Show/check network instructions and diagnostics
Enter class code; optional name or student ID
Backend checks code, class state, blocking and participant limit

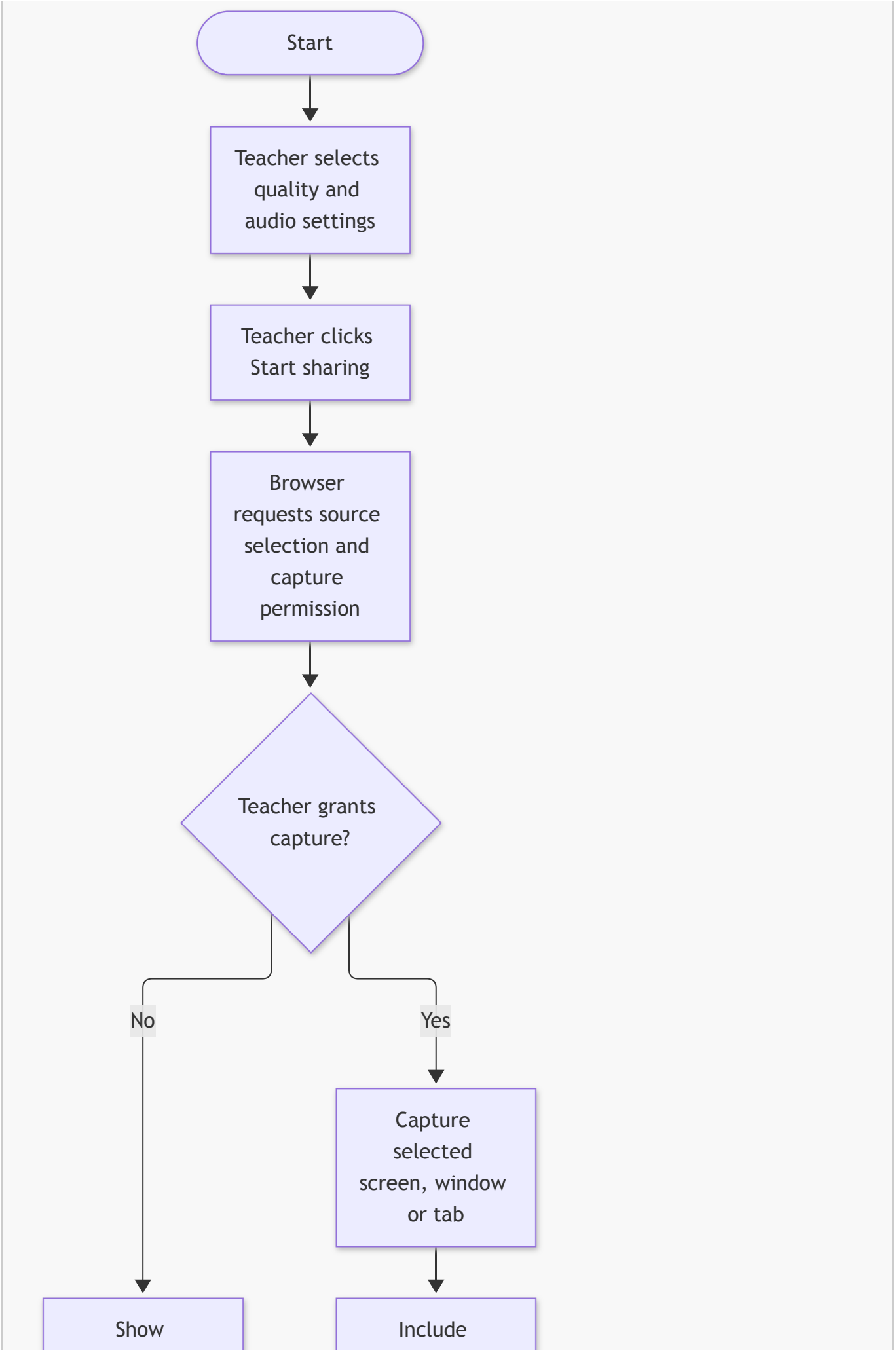

Join allowed?
Yes
No
Manual approval configured?
Yes
No
Display pending request to teacher
Show invalid code, full, blocked or ended message
Teacher approves?
Yes
No

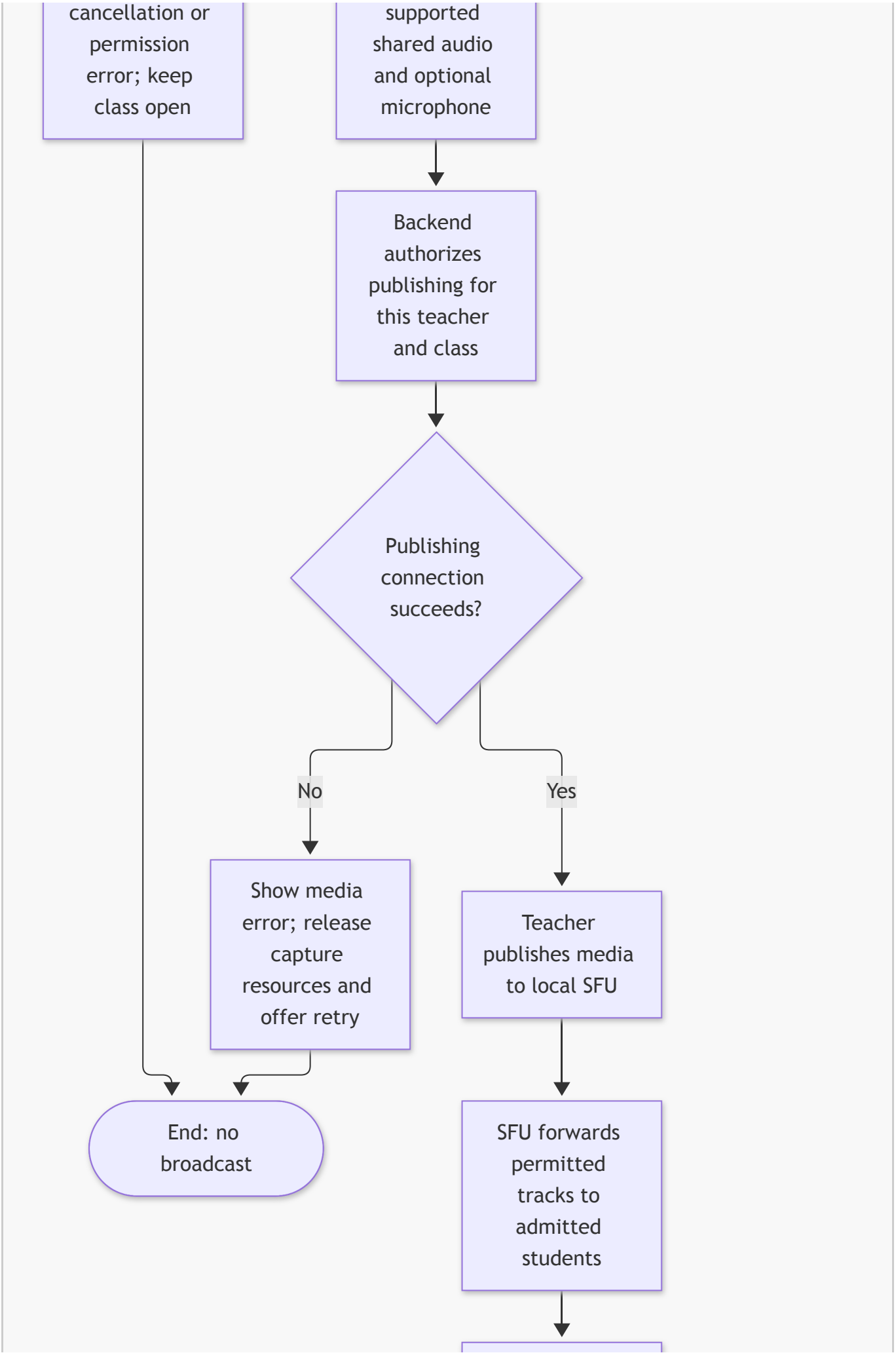


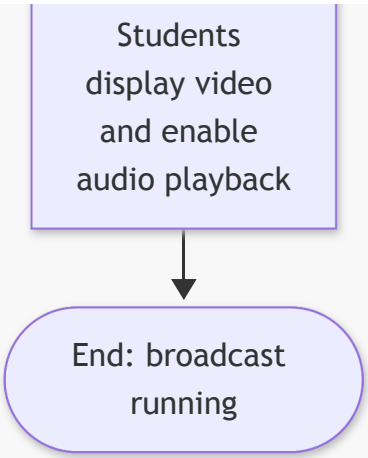
The diagram assumes the teacher makes a decision on a pending request; cancellation/timeouts require a final design. Names and class codes are not strong identity proof. Blocking persistence must be defined, particularly if students join anonymously.

Flow 06 — Start screen and audio broadcasting

Covers T02–T04, T07, S03–S04. WebRTC carries the real-time media. Signaling establishes the connection. The SFU receives teacher media and forwards it to students; the database stores application records, not the live video stream.


Start
Teacher selects quality and audio settings
Teacher clicks Start sharing
Browser requests source selection and capture permission
Teacher grants capture?
No
Yes
Capture selected screen, window or tab
Show
Include

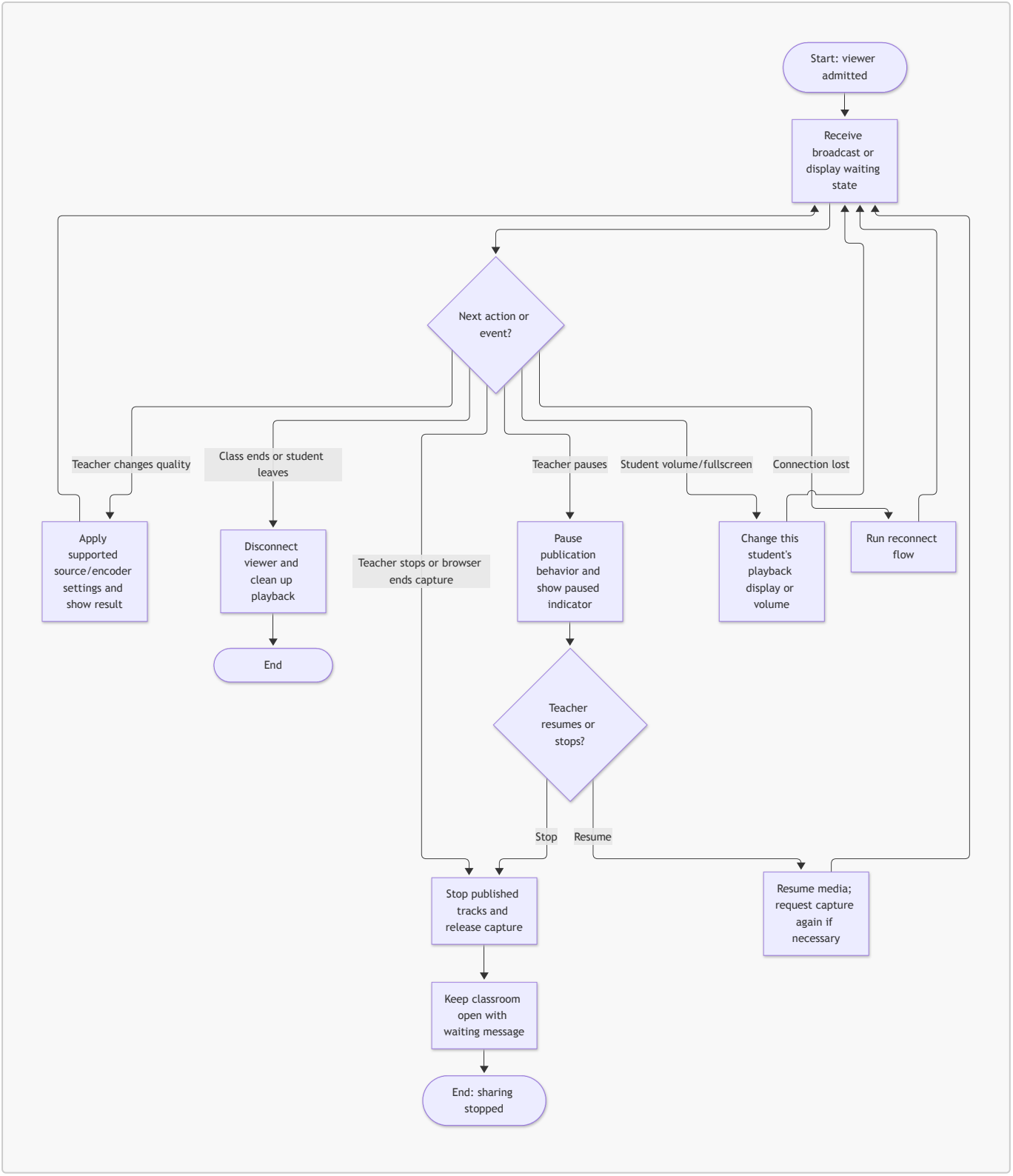

cancellation or permission error; keep class open
supported shared audio and optional microphone
Backend authorizes publishing for this teacher and class
Publishing connection succeeds?
No
Yes
Show media error; release capture resources and offer retry
Teacher publishes media to local SFU
End: no broadcast
SFU forwards permitted tracks to admitted students



Capture requires the browser-supported secure context and user permission specified in the source. Audio support depends on operating system, browser and source selection. If audio is unavailable, show that limitation rather than claiming it is being shared. Agree which combinations satisfy the requirement.

Flow 07 — View, pause, resume and stop sharing

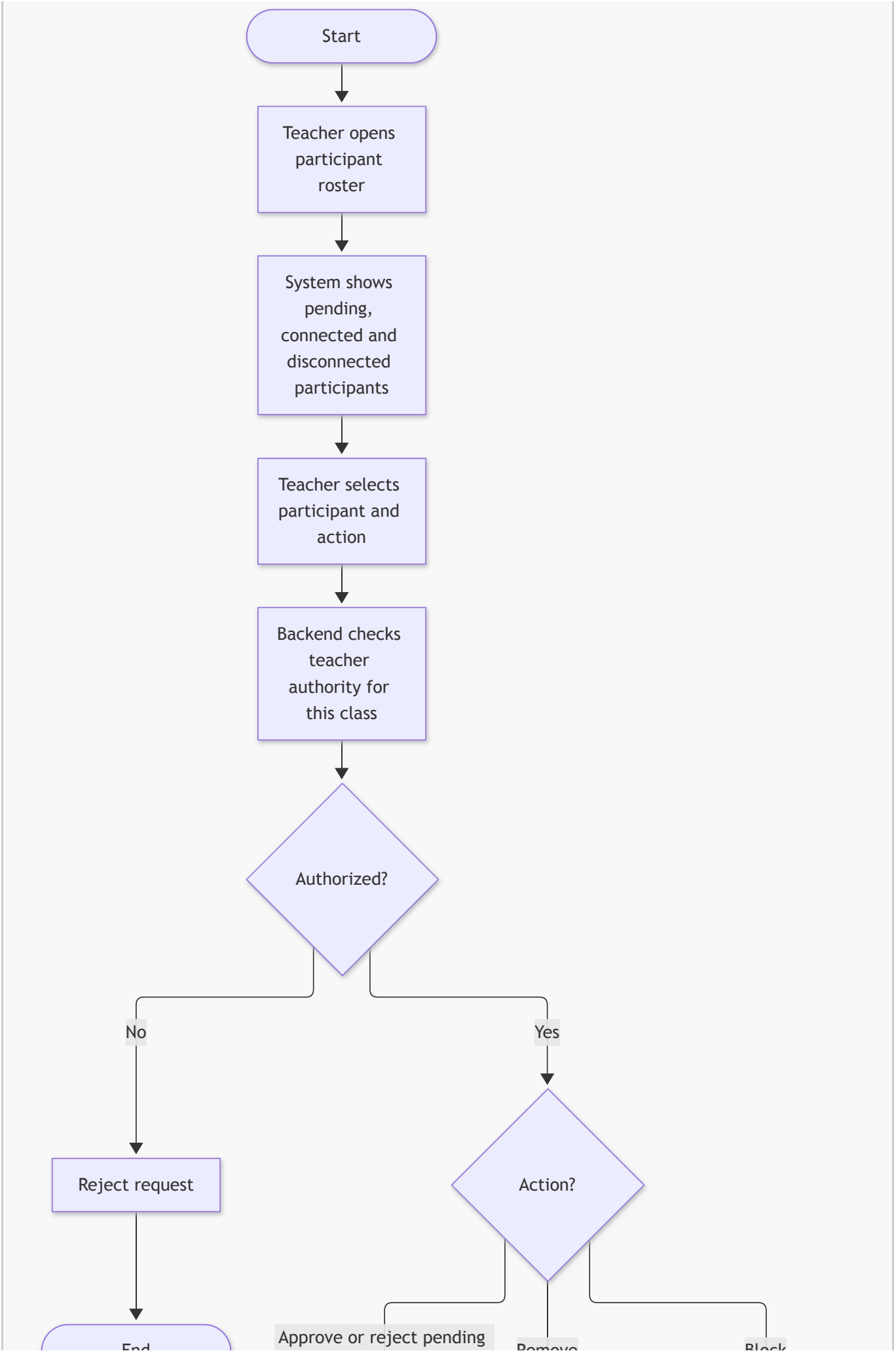
Covers T02, T07, S03–S05, S07. A student changes their own playback controls. A teacher changes the published stream. Stopping sharing is different from ending the classroom.

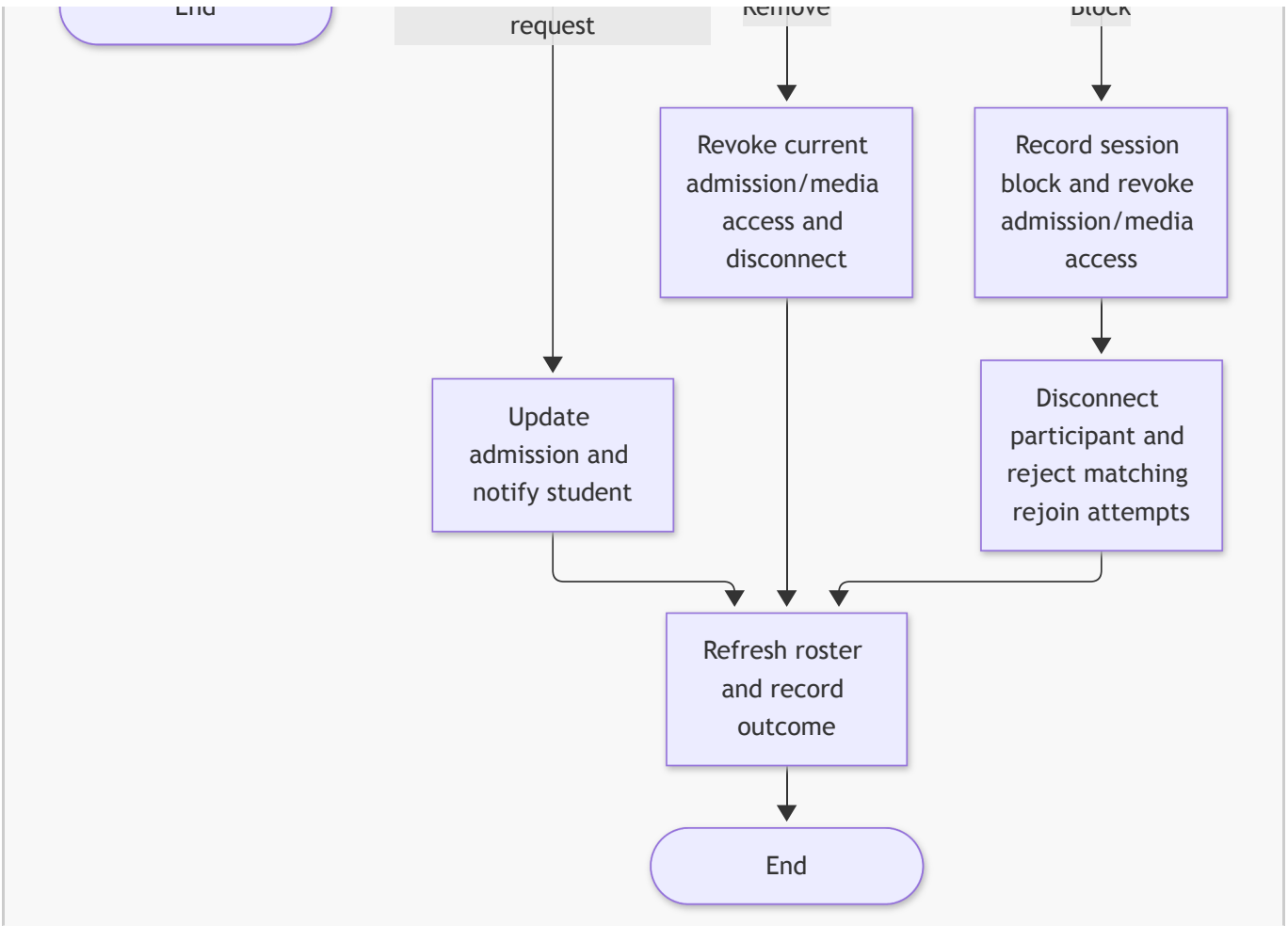


Unresolved: whether pause freezes the last frame or shows a placeholder, whether audio pauses, and whether changing resolution requires new capture. Proposed behavior: clear paused indicator and no continuing live audio. The stop branch does not automatically restore the class policy unless the class is ended or restrictions are separately disabled.

Flow 08 — Teacher monitors, removes or blocks a participant

Covers T05–T06 and classroom access control. Monitoring here means connected participants and connection status. Viewing student screens, remote control and monitoring all their activity are not specified.

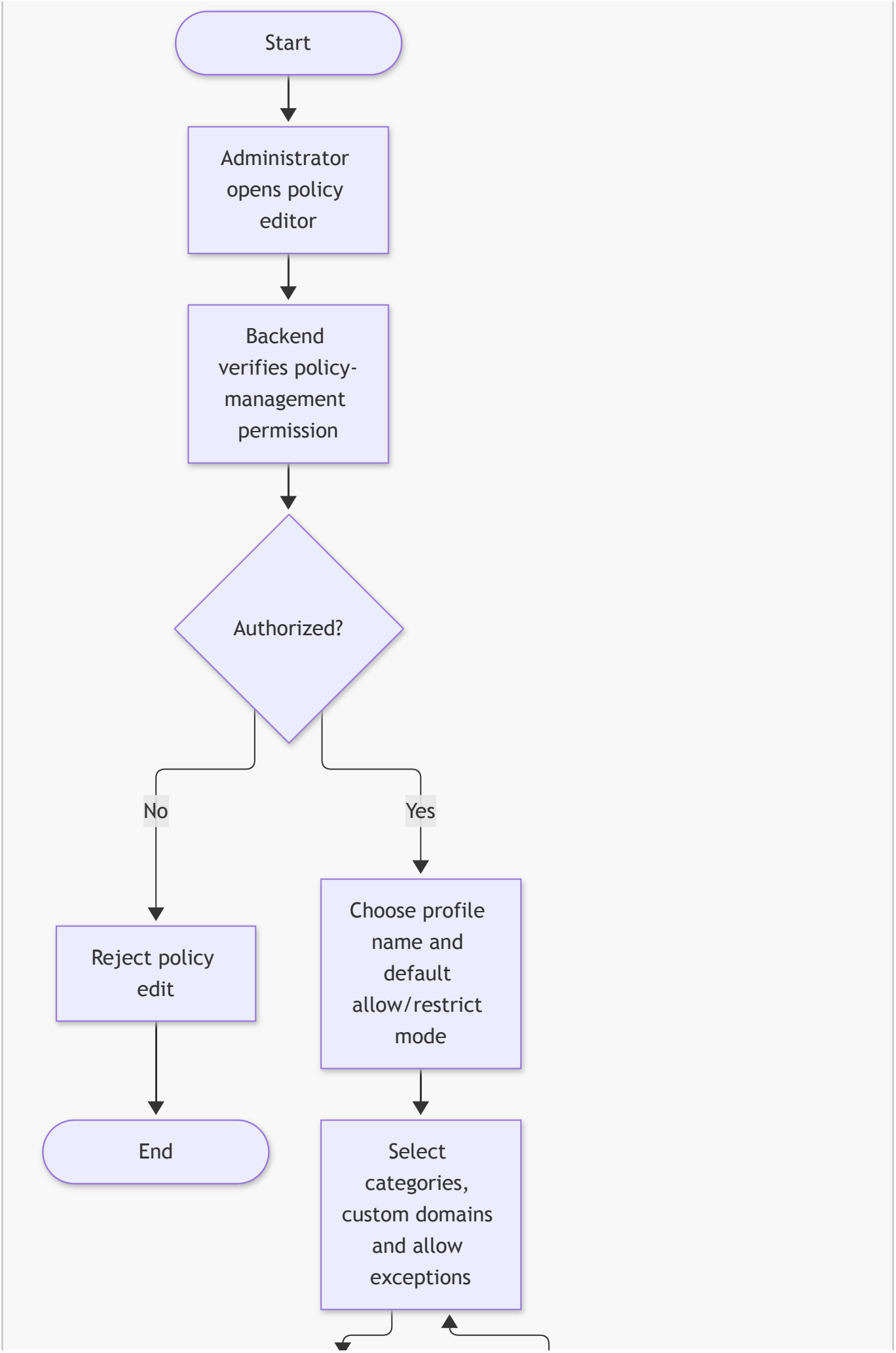

Start
Teacher opens participant roster
System shows pending, connected and disconnected participants
Teacher selects participant and action
Backend checks teacher authority for this class
Authorized?
No
Yes
Reject request
Action?
Approve or reject pending

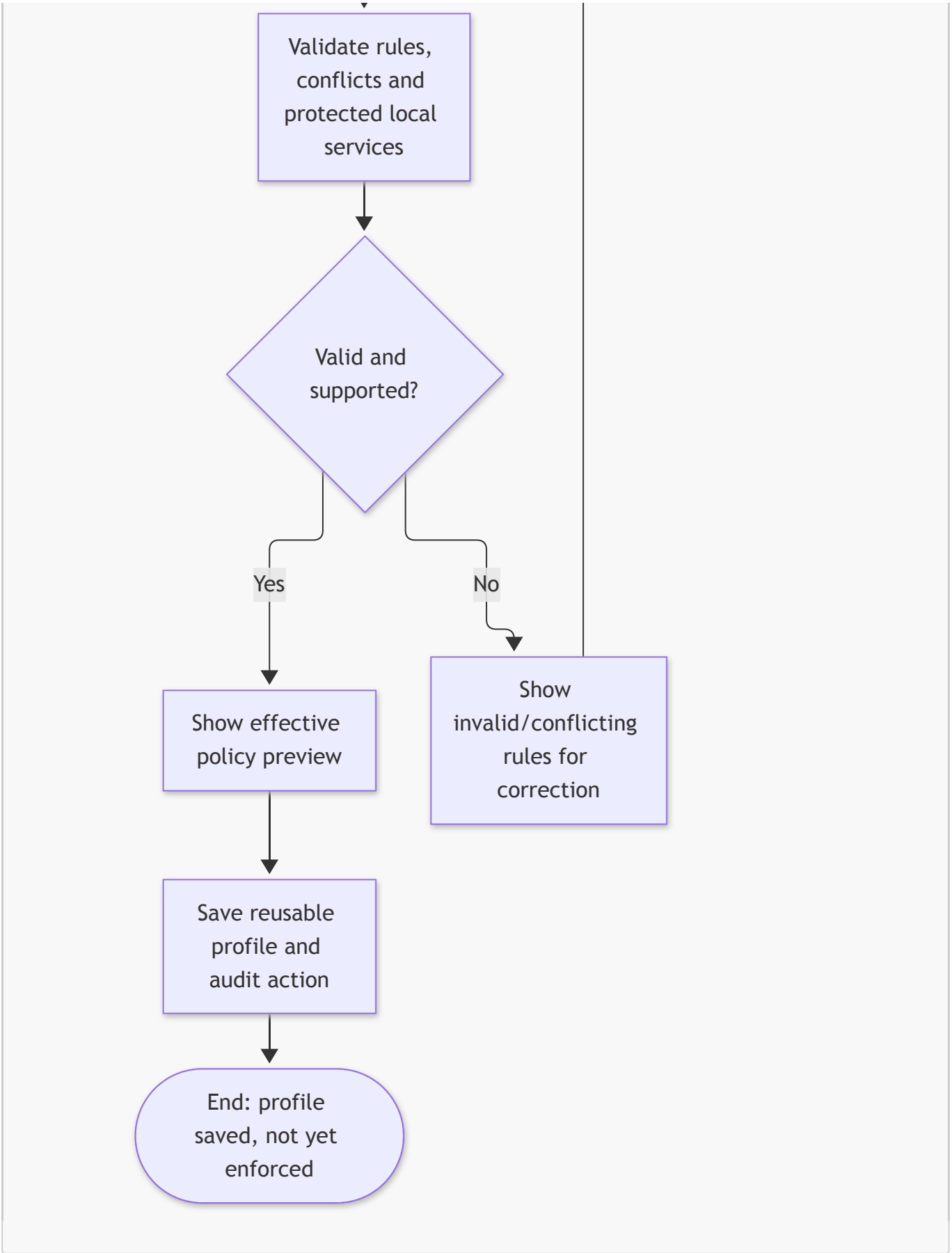


Proposed distinction: removal ends current participation; blocking also prevents rejoin for the agreed identity and duration. The source requires both actions, but identity and block lifetime need clarification. A browser-only anonymous application cannot guarantee recognition after all browser identity is cleared.

Flow 09 — Administrator creates a policy profile

Covers NAC02–NAC04, NAC07–NAC08, NAC16–NAC18. A profile is a reusable configuration, such as Normal Class, Programming Lab or Exam Mode. Saving one does not apply it to the network.


Start
Administrator opens policy editor
Backend verifies policy-management permission
Authorized?
No
Yes
Reject policy edit
End
Choose profile name and default allow/restrict mode
Select categories, custom domains and allow exceptions

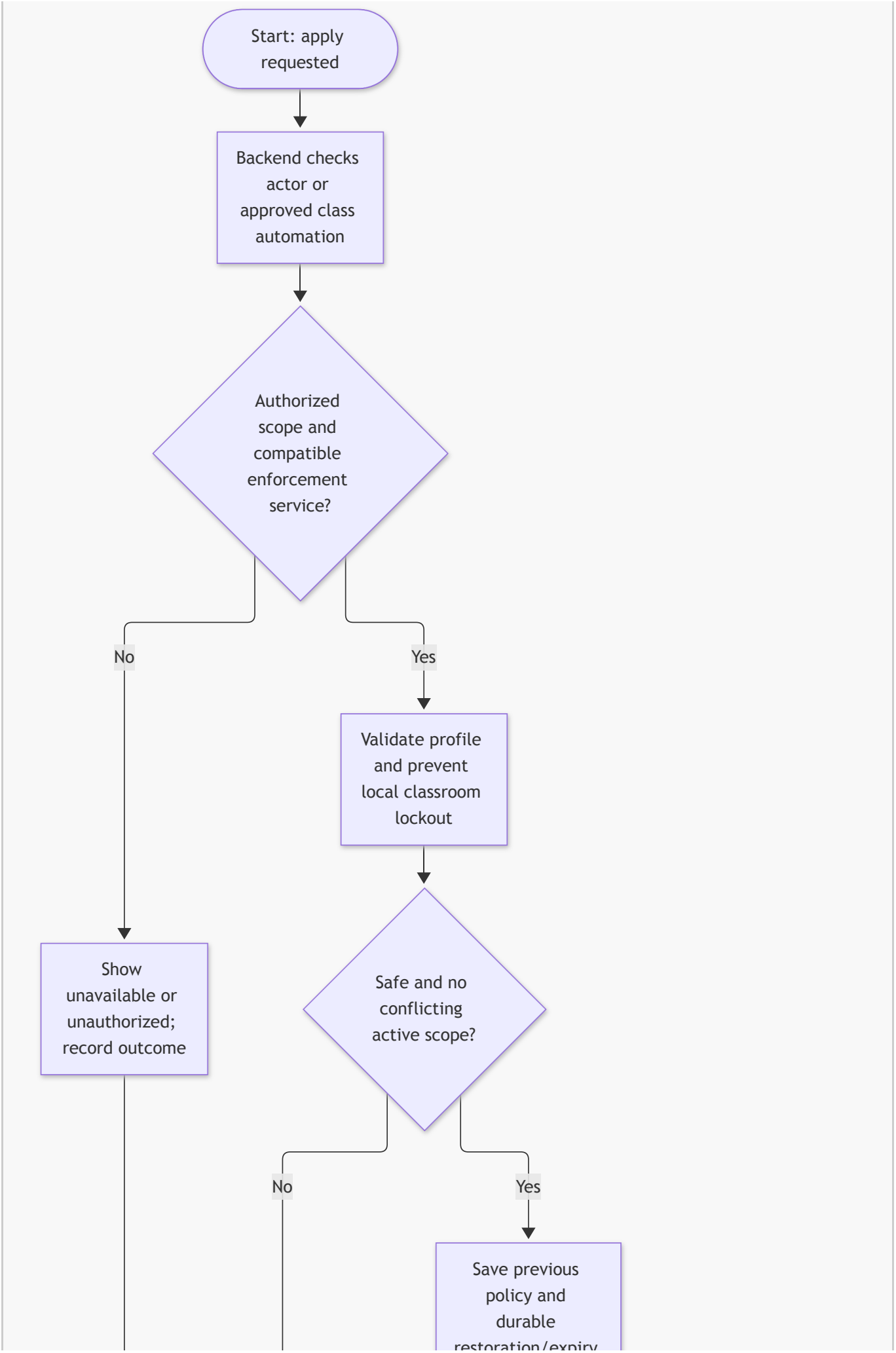


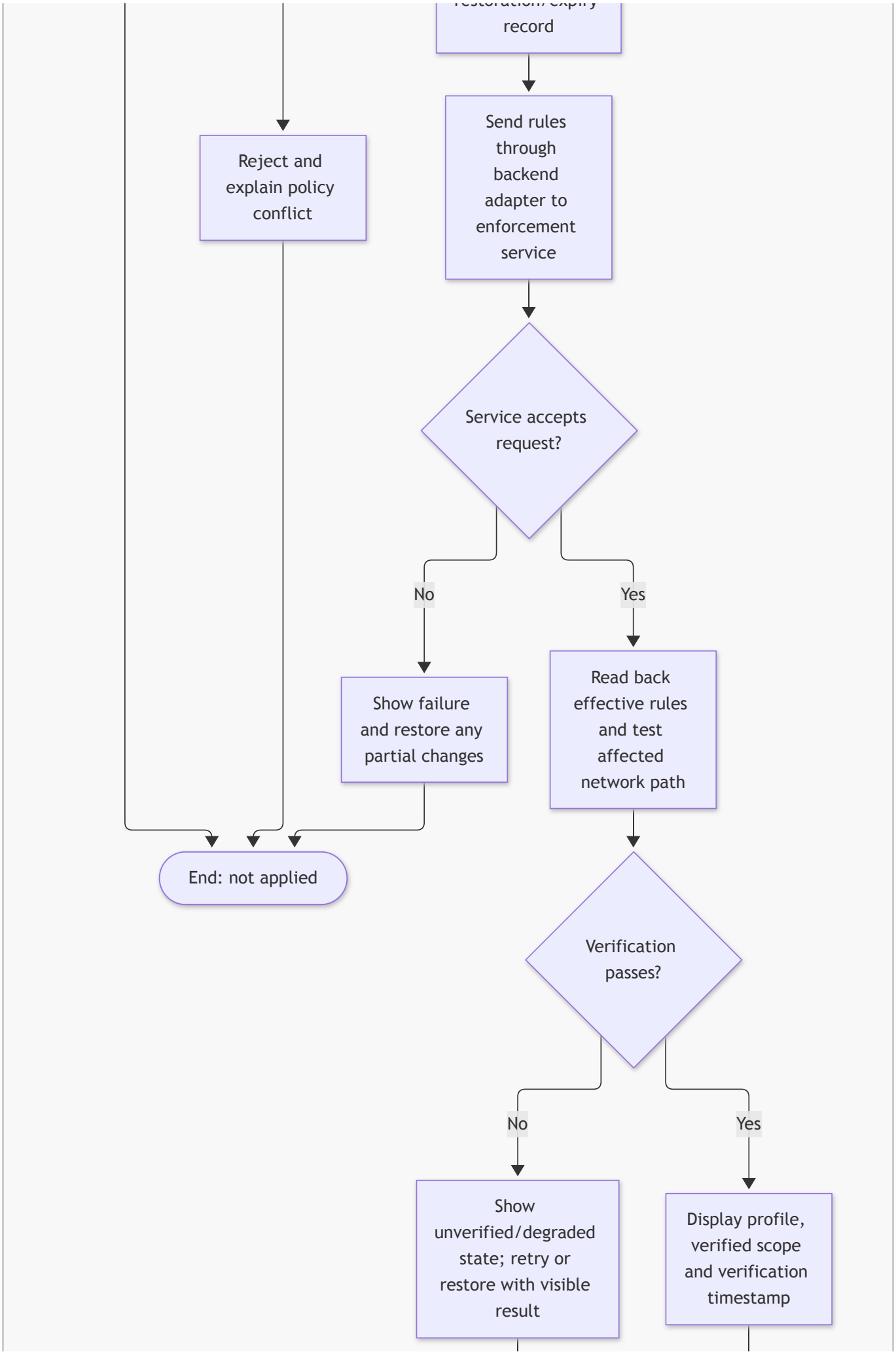
Allow-by-default: public destinations are permitted unless a rule restricts them. **Restricted-by-default:** only explicitly approved destinations are permitted within the adapter's supported scope. Specify exception precedence and domain/subdomain matching before implementation. Neither mode alone proves protection against VPNs or alternate DNS.

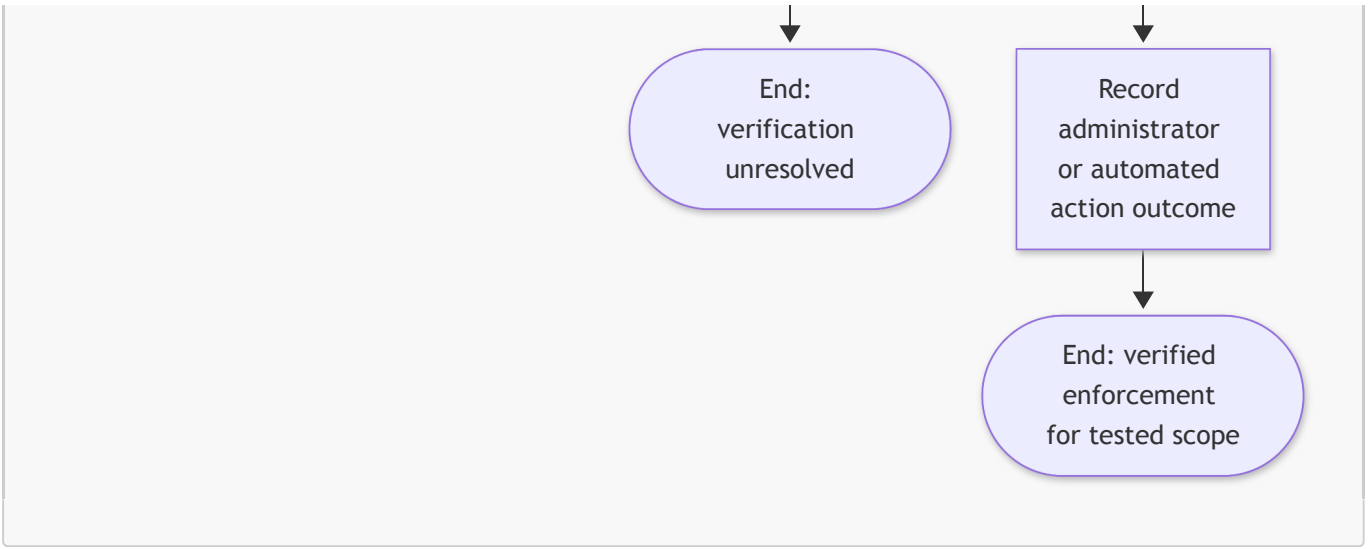
Categories are maintained domain collections, not an automatic guarantee of identifying every AI tool. The category checkbox preview in the pasted requirements is an illustration, not an active policy.

Flow 10 — Apply and verify restrictions

Covers NAC01, NAC05, NAC10–NAC11, NAC16–NAC19; CN10/OR12. Trigger: a class starts with an administrator-approved profile, or an authorized administrator requests application. The teacher's start event can trigger preapproved policy behavior without giving the teacher unrestricted policy-edit privileges.


Start: apply requested
Backend checks actor or approved class automation
Authorized scope and compatible enforcement service?
No
Yes
Validate profile and prevent local classroom lockout
Show unavailable or unauthorized; record outcome
Safe and no conflicting active scope?
No
Yes
Save previous policy and durable


record
Send rules through backend adapter to enforcement service
Reject and explain policy conflict
Service accepts request?
No
Yes
Show failure and restore any partial changes
Read back effective rules and test affected network path
End: not applied
Verification passes?
No
Yes
Show unverified/degraded state; retry or restore with visible result
Display profile, verified scope and verification timestamp

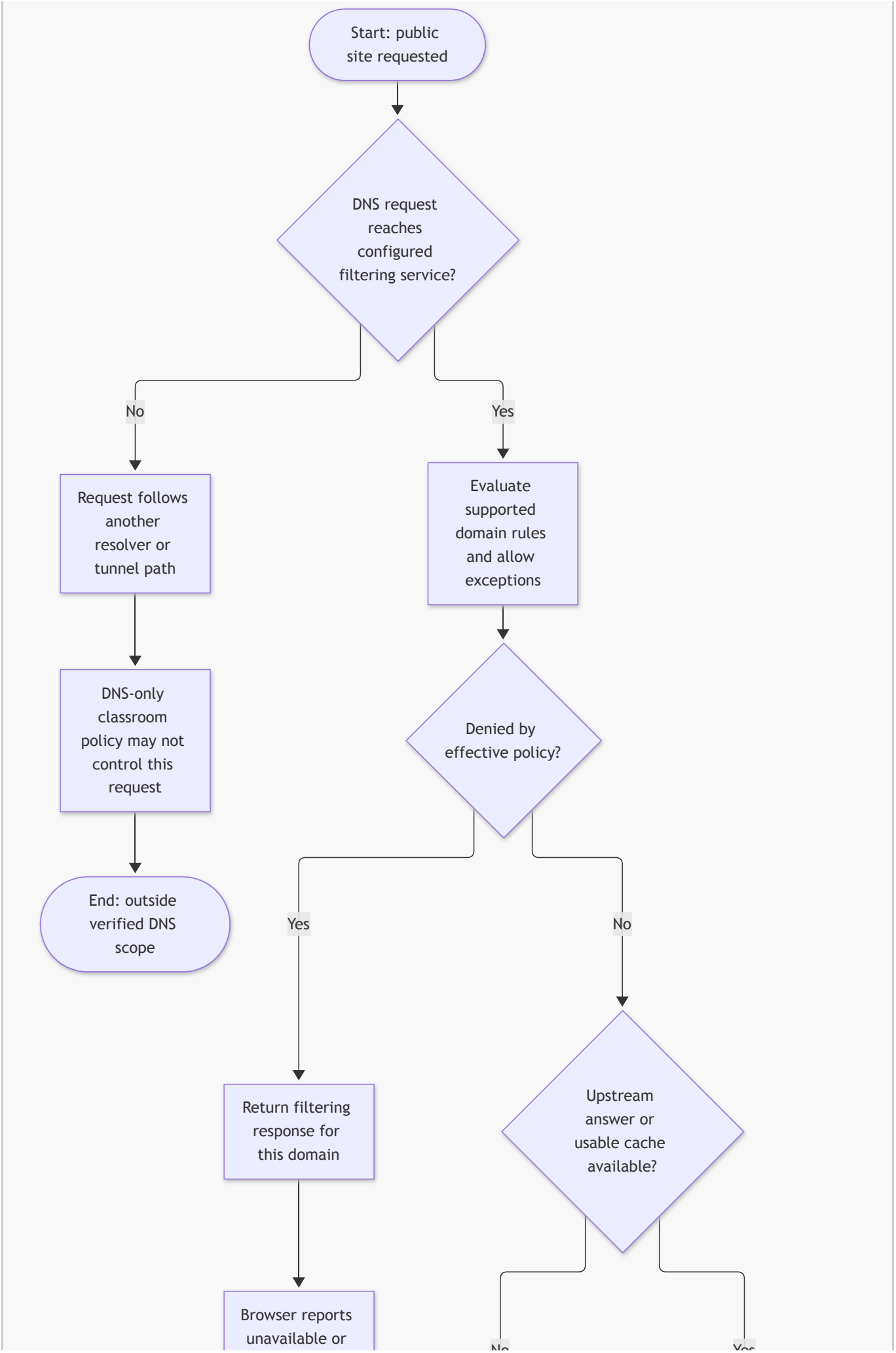


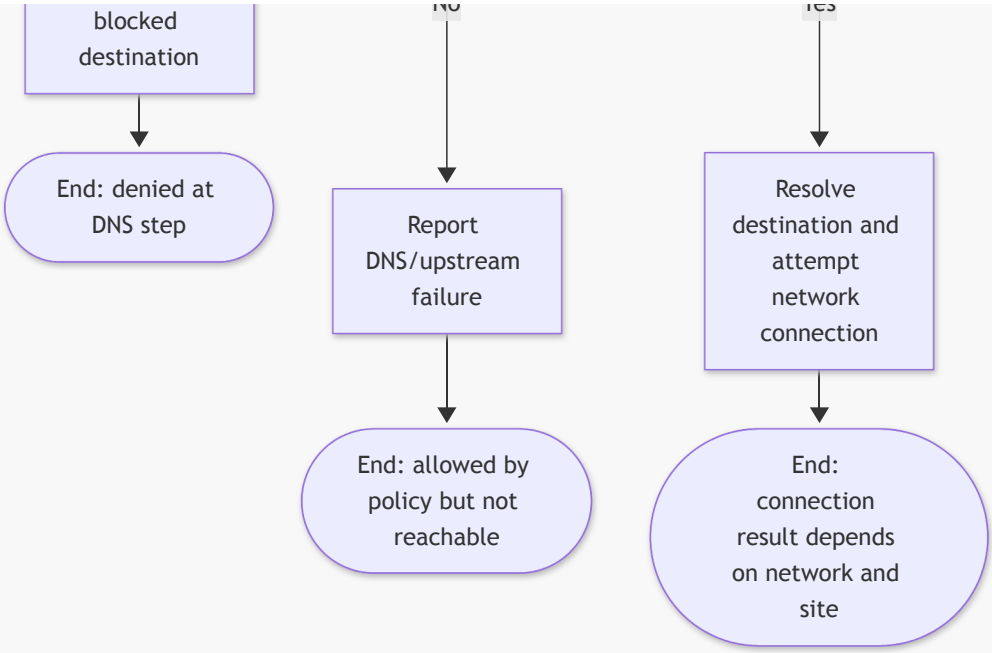
Saving a baseline durably and refusing conflicting scope operations are proposed implementation safeguards. The requirement is reliable apply/status/restore behavior; the storage mechanism is a design choice.

Three different results: profile saved; service accepted changes; restrictions verified on the tested path. Display them accurately. A backend DNS test alone does not establish that every student is using the configured resolver.

Flow 11 — A student requests a public website

This explains what the enforcement component does. It is separate from the student watching the local broadcast. It also explains the supplied DNS-filtering limitations.

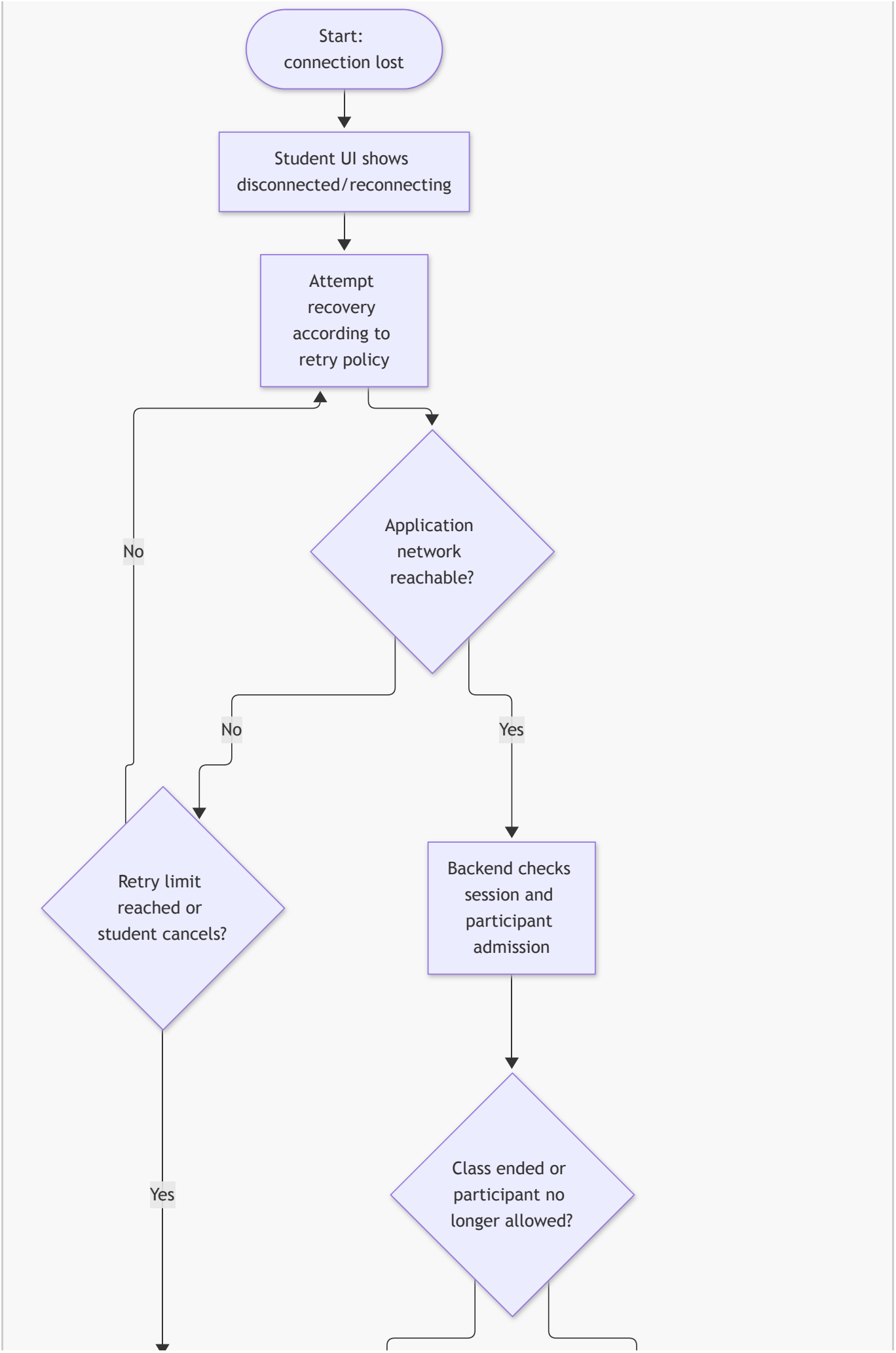

Start: public site requested
DNS request reaches configured filtering service?
No
Yes
Request follows another resolver or tunnel path
Evaluate supported domain rules and allow exceptions
DNS-only classroom policy may not control this request
Denied by effective policy?
End: outside verified DNS scope
Yes
No
Return filtering response for this domain
Upstream answer or usable cache available?
Browser reports unavailable or
No
Yes

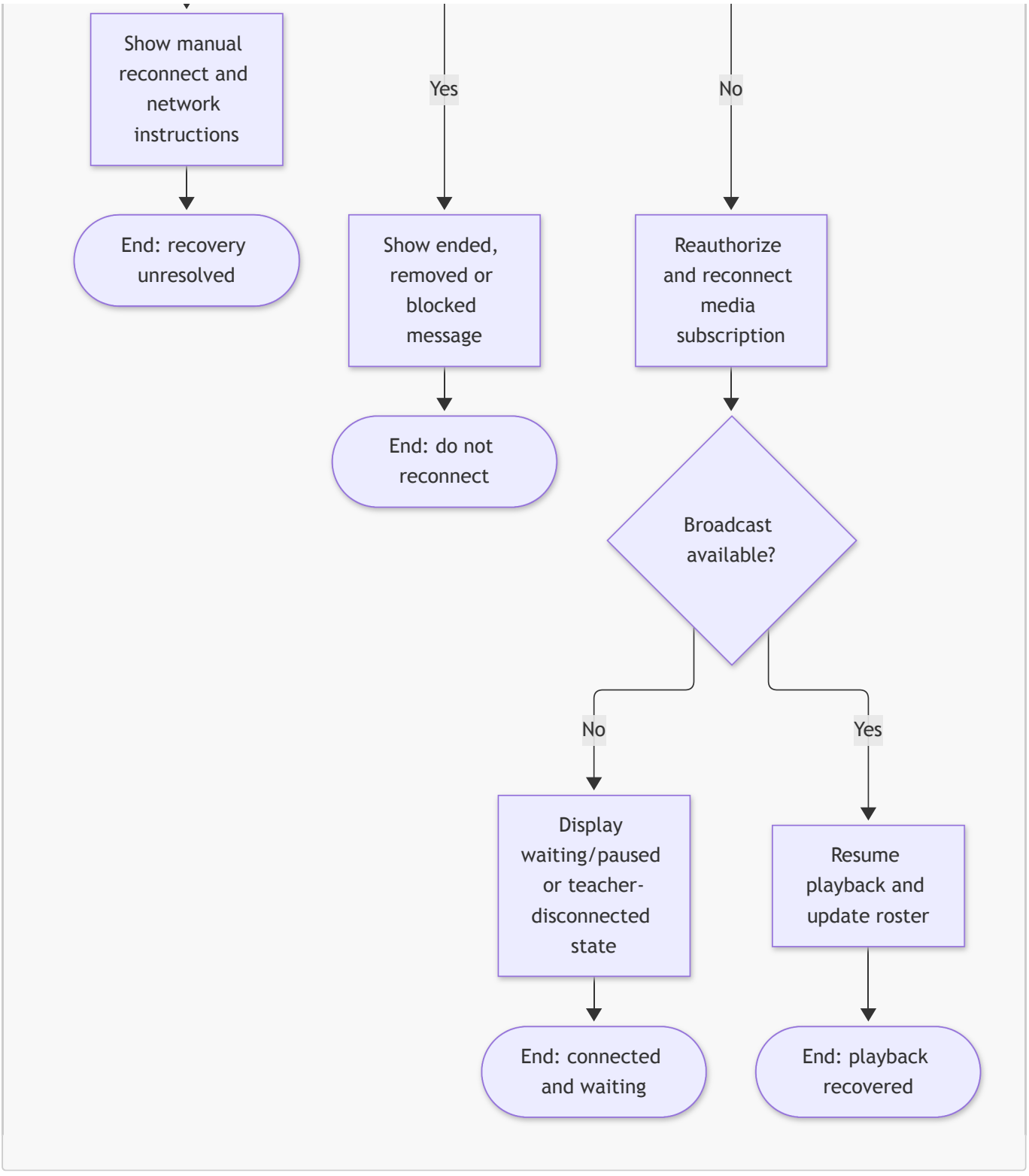


This diagram is scoped to DNS lookups. Cached addresses, existing connections, direct IP access and offline agents need separate treatment; policy changes may not instantly terminate existing website connections. A gateway can enforce additional restrictions only where supported and authorized. Do not promise a custom block page for every HTTPS website.

Flow 12 — Connection loss and reconnection

Covers S06–S07 and teacher connection status. The source requires reconnection but does not define timeout/retry limits or teacher-offline behavior.

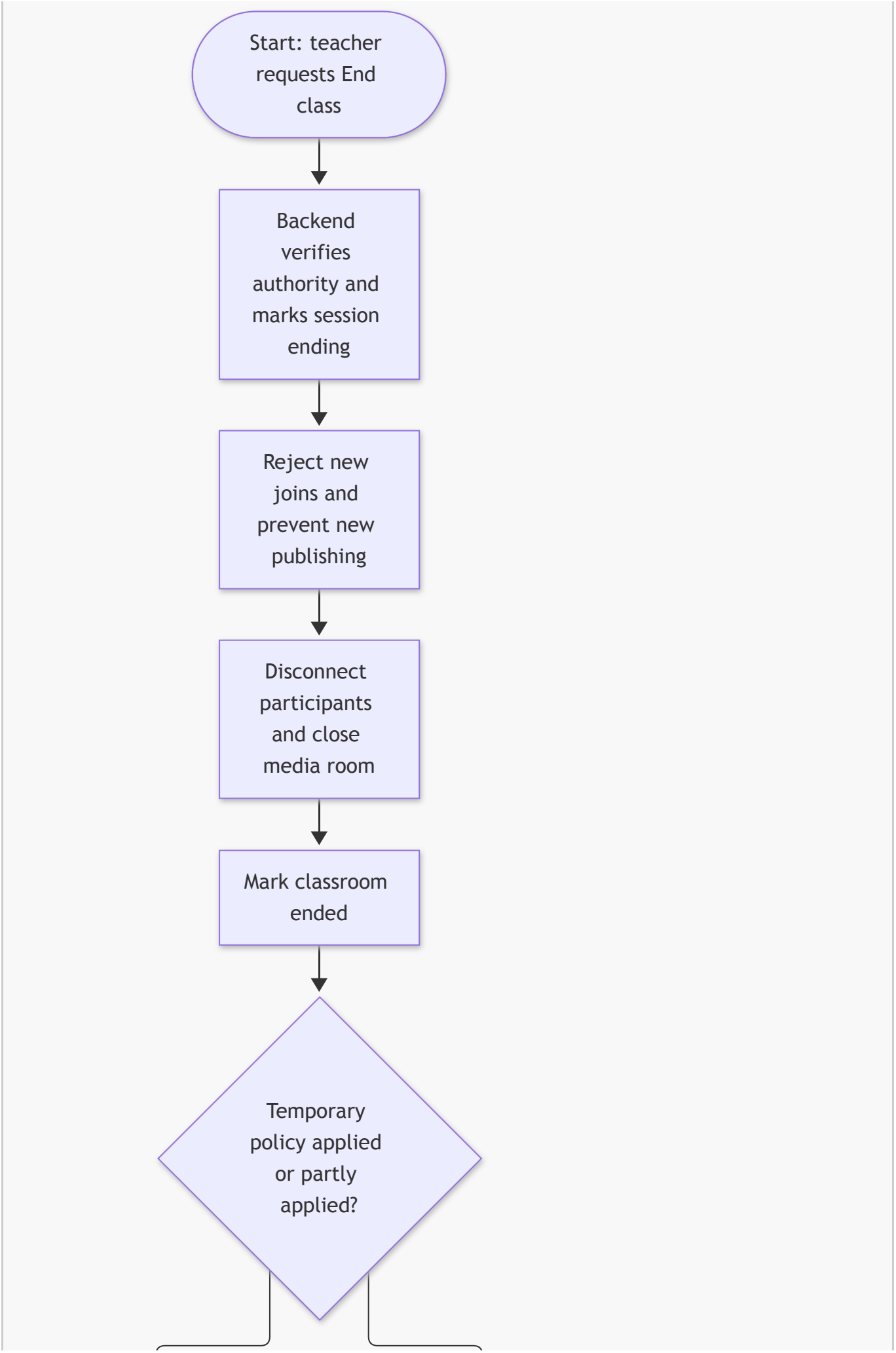

Start:
connection lost
Student UI shows
disconnected/reconnecting
Attempt
recovery
according to
retry policy
Application
network
reachable?
No
No
Yes
Retry limit
reached or
student cancels?
Backend checks
session and
participant
admission
Yes
Class ended or
participant no
longer allowed?

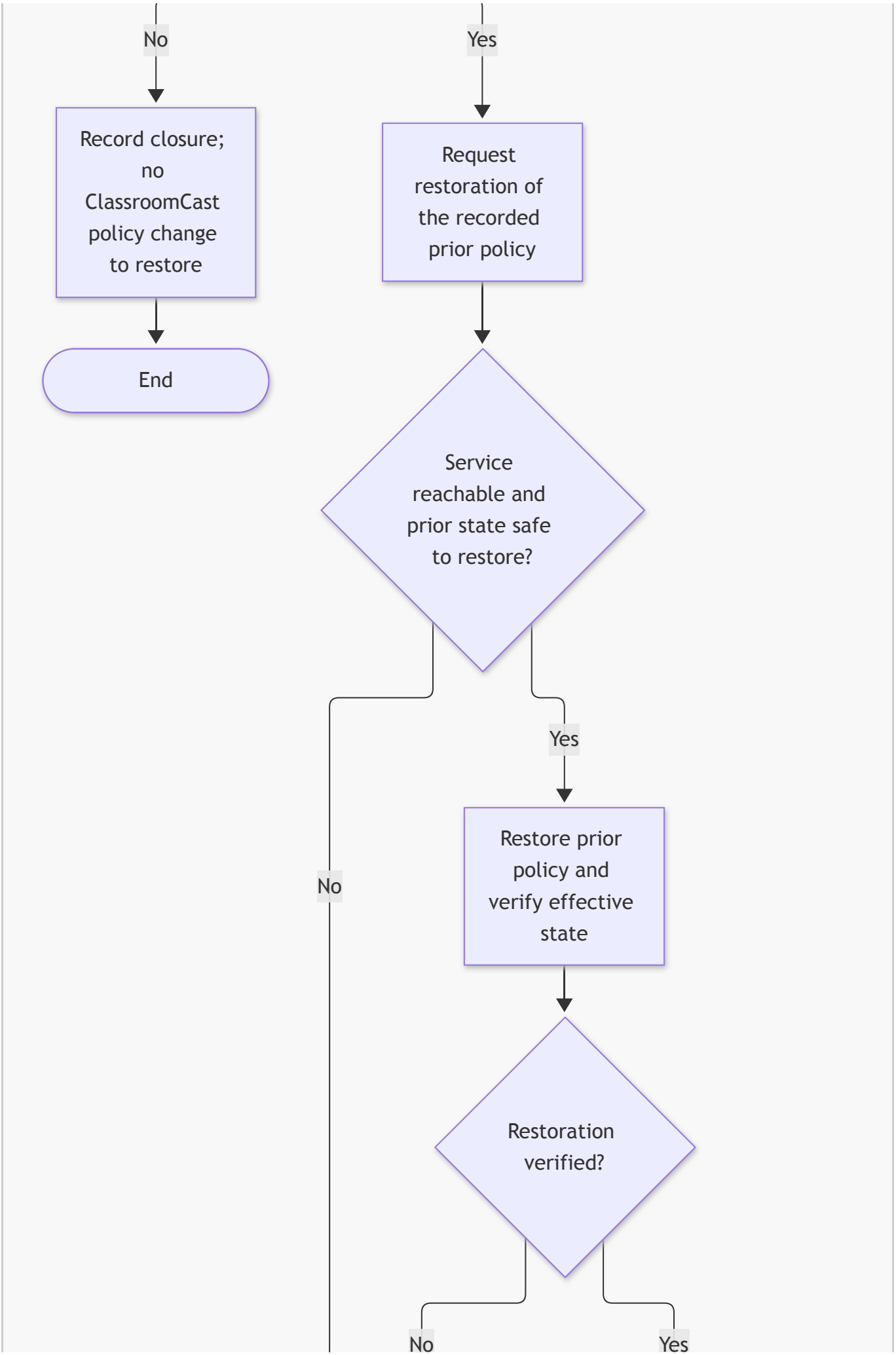


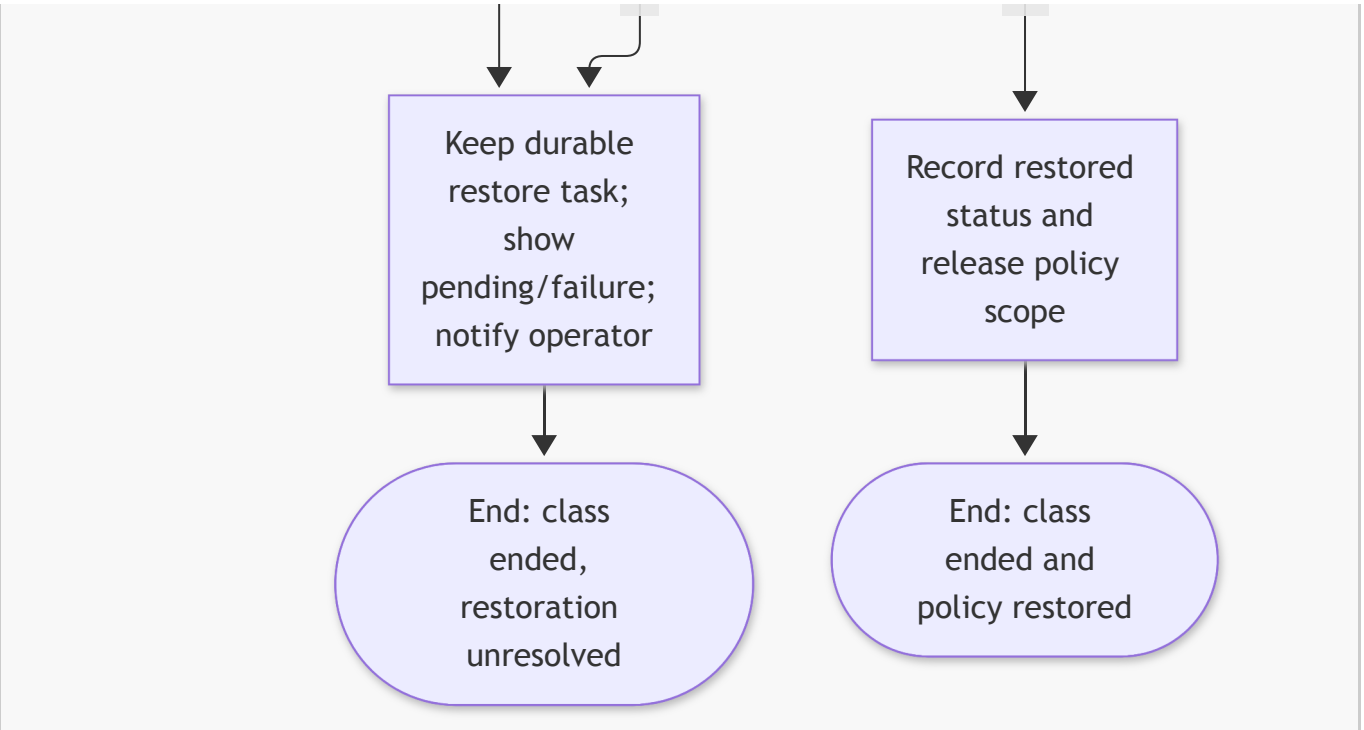
A temporary teacher connection failure should not silently mean that network restrictions are restored or left forever. Proposed behavior: keep explicit class state, use visible recovery status, and retain an independent policy expiry mechanism. Agree the teacher-disconnect grace period before implementation.

Flow 13 — End a classroom and restore policy

Covers NAC06, NAC10–NAC11 and classroom end/disconnect requirements. Closing student access and restoring network policy are separate outcomes; a failed restore must remain visible even after the class has ended.


Start: teacher requests End class
Backend verifies authority and marks session ending
Reject new joins and prevent new publishing
Disconnect participants and close media room
Mark classroom ended
Temporary policy applied or partly applied?

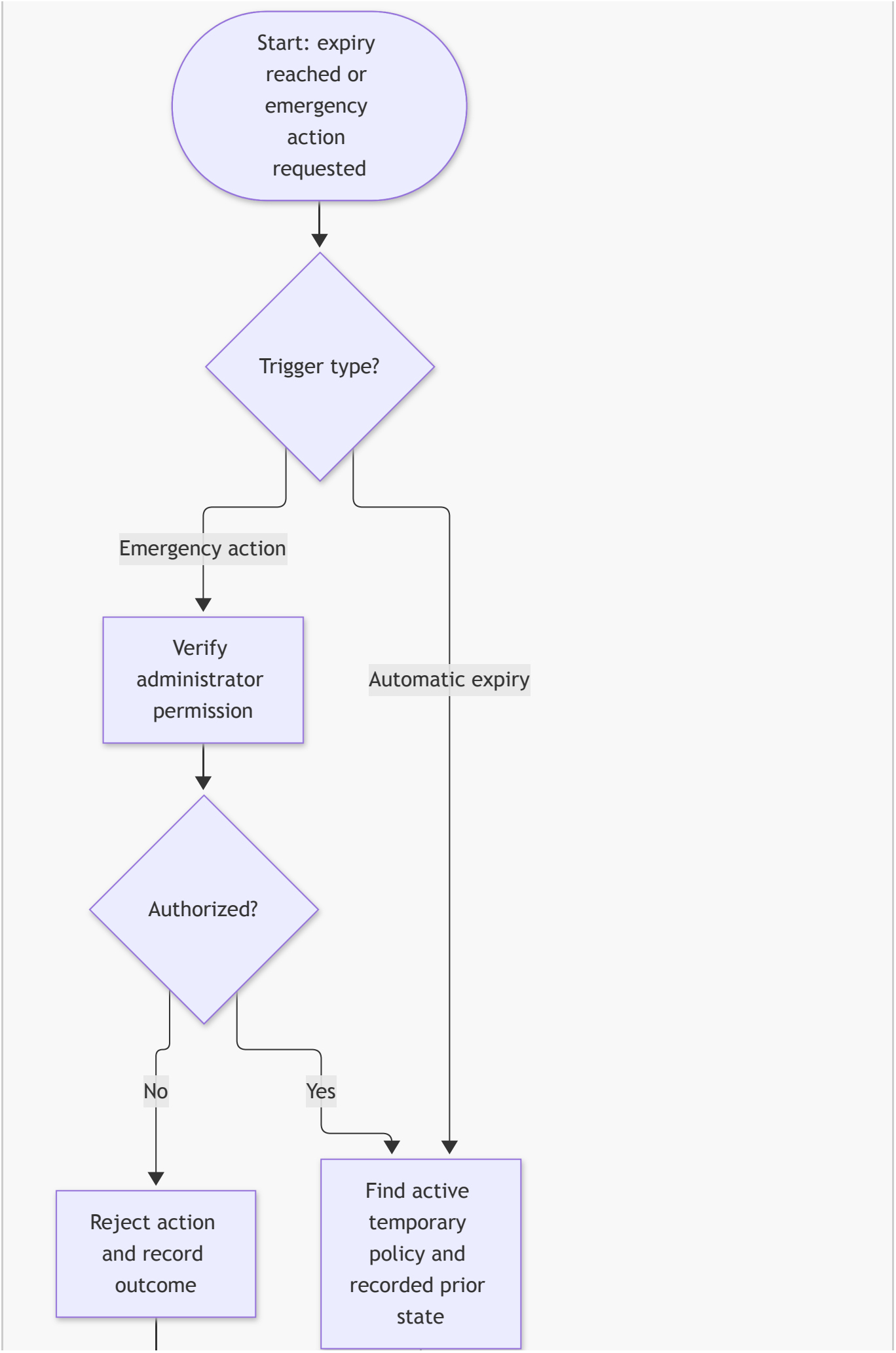

No
Yes
Record closure; no ClassroomCast policy change to restore
Request restoration of the recorded prior policy
End
Service reachable and prior state safe to restore?
Yes
No
Restore prior policy and verify effective state
Restoration verified?
No
Yes

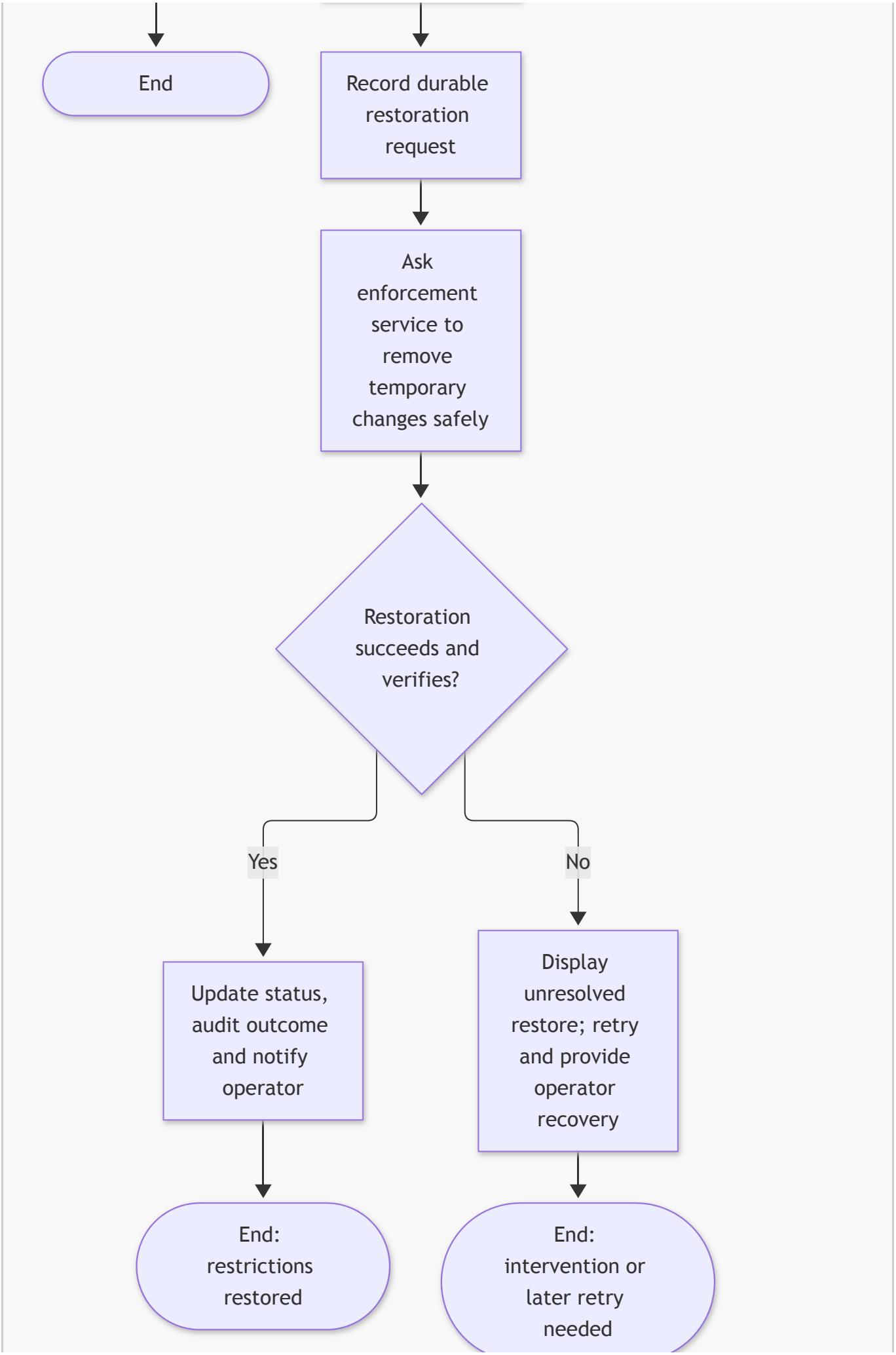


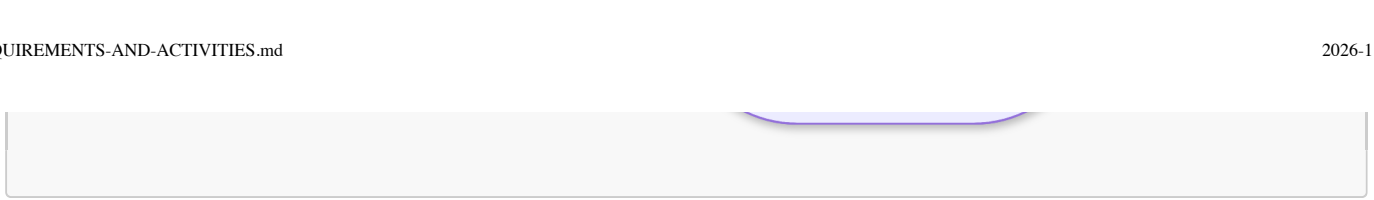
Proposed safeguard: detect unrelated configuration changes before overwriting a saved snapshot. A simple unrestricted “allow everything” operation is not necessarily correct restoration. Preserve protections that existed before ClassroomCast started the class.

Flow 14 — Temporary expiry and emergency disable

Covers NAC12–NAC13 and recovery requirements. Expiry should operate without a teacher browser remaining open. Proposed durable expiry records and a recovery worker support that requirement.


Start: expiry reached or emergency action requested
Trigger type?
Emergency action
Automatic expiry
Verify administrator permission
Authorized?
No
Yes
Reject action and record outcome
Find active temporary policy and recorded prior state

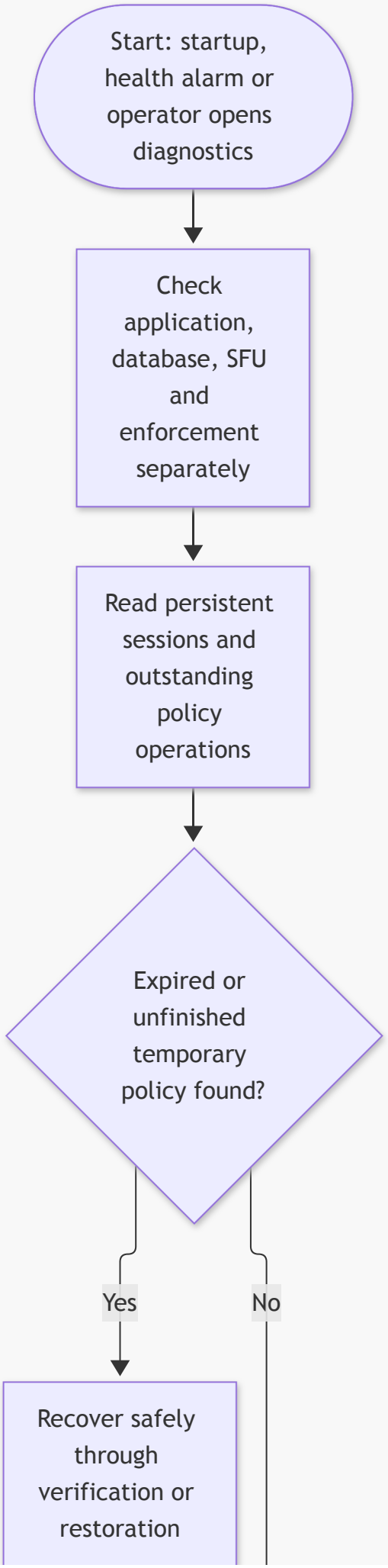

End
Record durable restoration request
Ask enforcement service to remove temporary changes safely
Restoration succeeds and verifies?
Yes
No
Update status, audit outcome and notify operator
Display unresolved restore; retry and provide operator recovery
End: restrictions restored
End: intervention or later retry needed

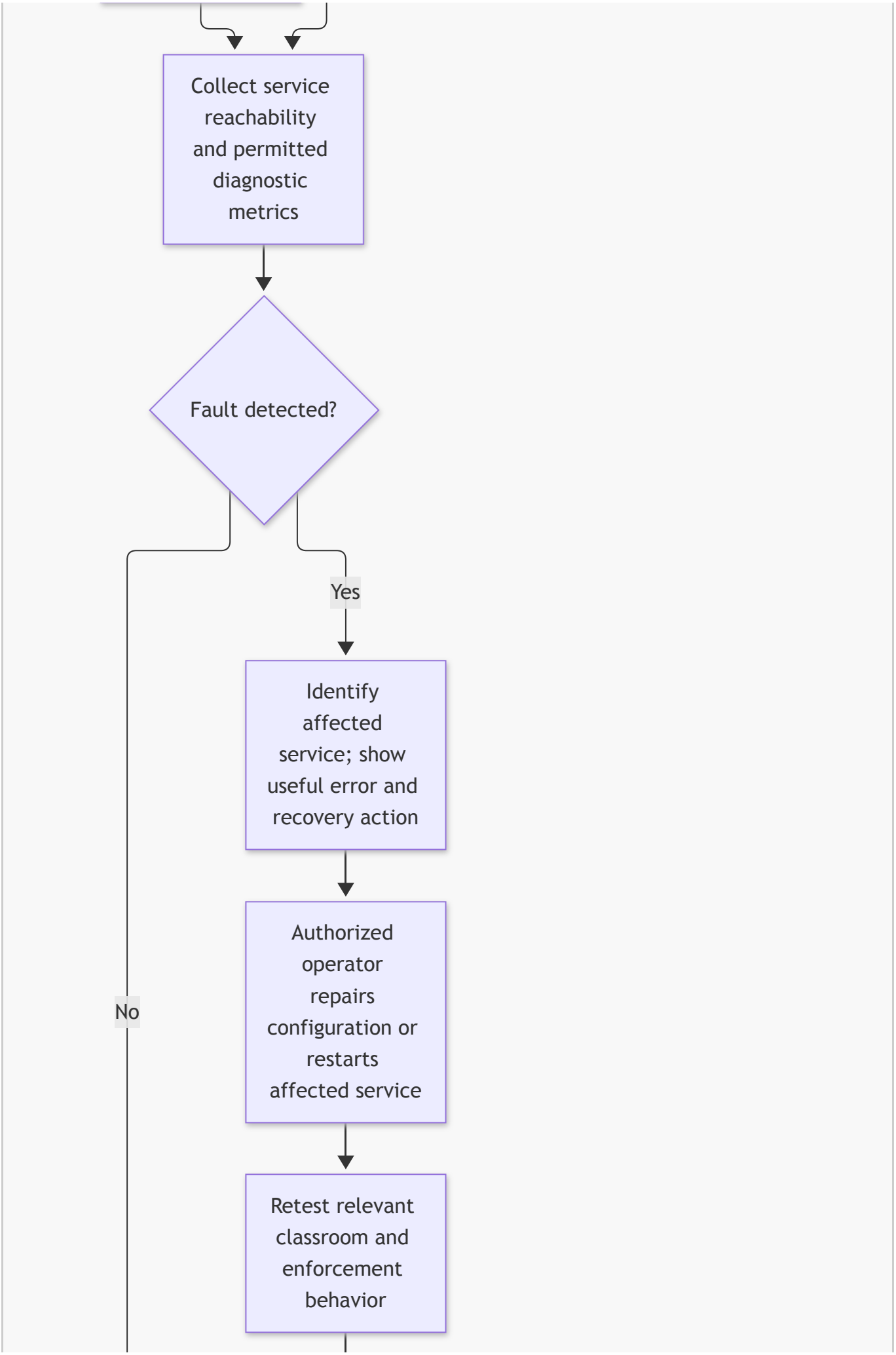


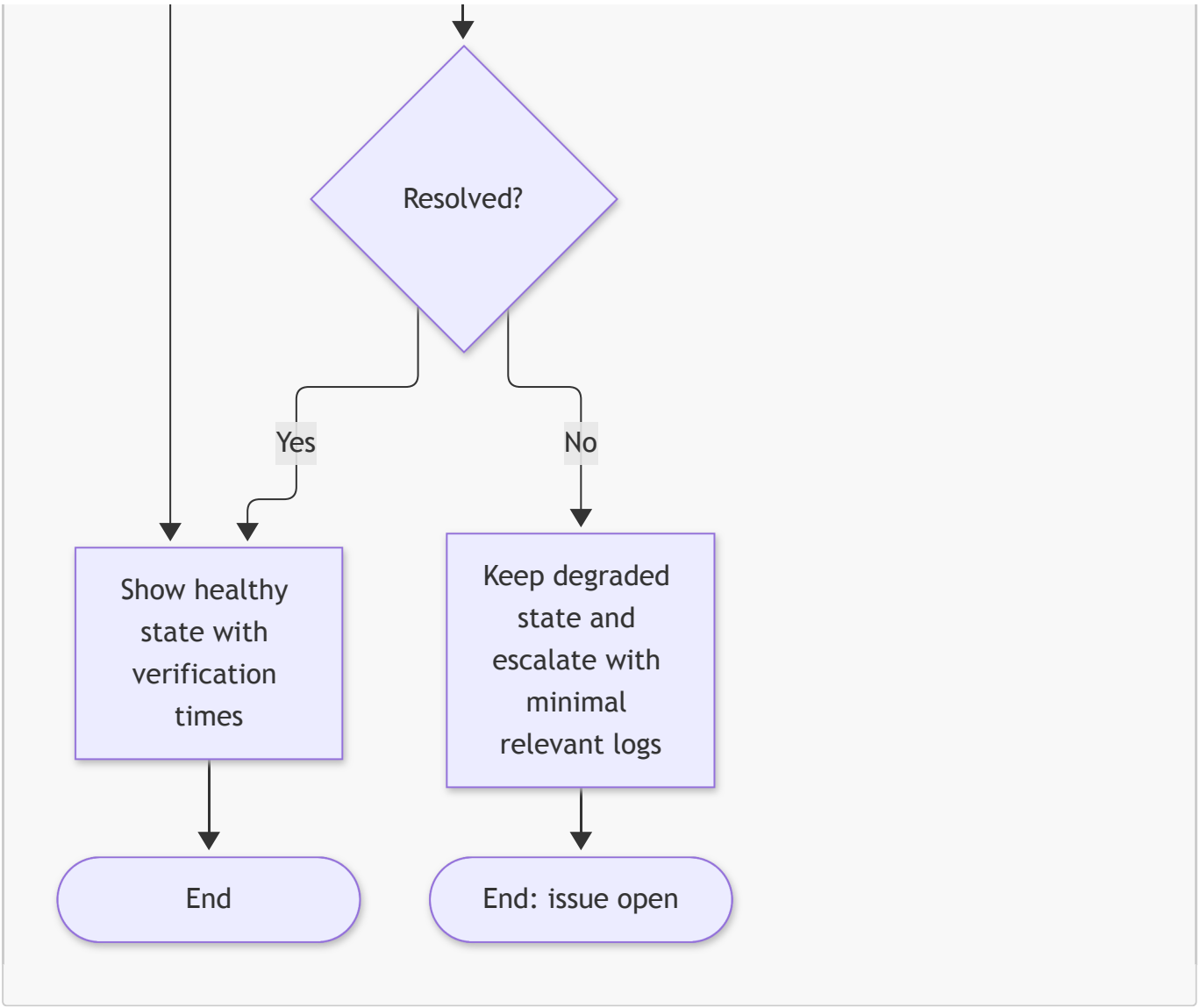
Restoration cannot be guaranteed while the enforcement service is unreachable. Independent service-side expiry may strengthen recovery where supported. Emergency disable should restore ClassroomCast's changes without disabling unrelated campus controls. Whether ending restrictions also ends the classroom is unspecified; proposed behavior is to keep these separate unless the operator ends the class.

Flow 15 — Health, restart and diagnosis

Covers local diagnostic page, health checks, logs and recovery; T09 is Should for expanded streaming diagnostics. Basic health/recovery obligations already exist in the architectural requirements.


Start: startup, health alarm or operator opens diagnostics
Check application, database, SFU and enforcement separately
Read persistent sessions and outstanding policy operations
Expired or unfinished temporary policy found?
Yes
No
Recover safely through verification or restoration

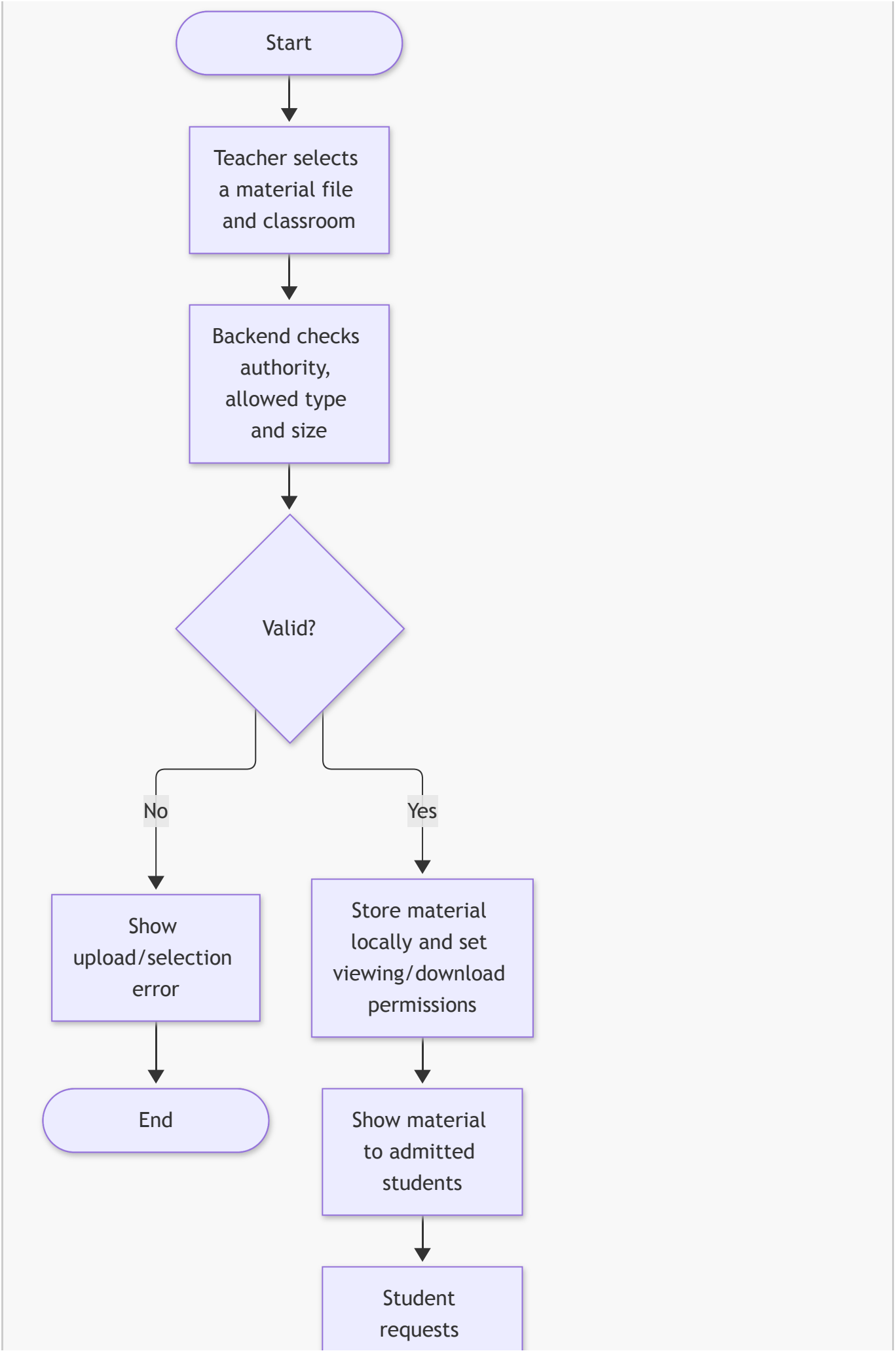

Collect service reachability and permitted diagnostic metrics
Fault detected?
Yes
No
Identify affected service; show useful error and recovery action
Authorized operator repairs configuration or restarts affected service
Retest relevant classroom and enforcement behavior

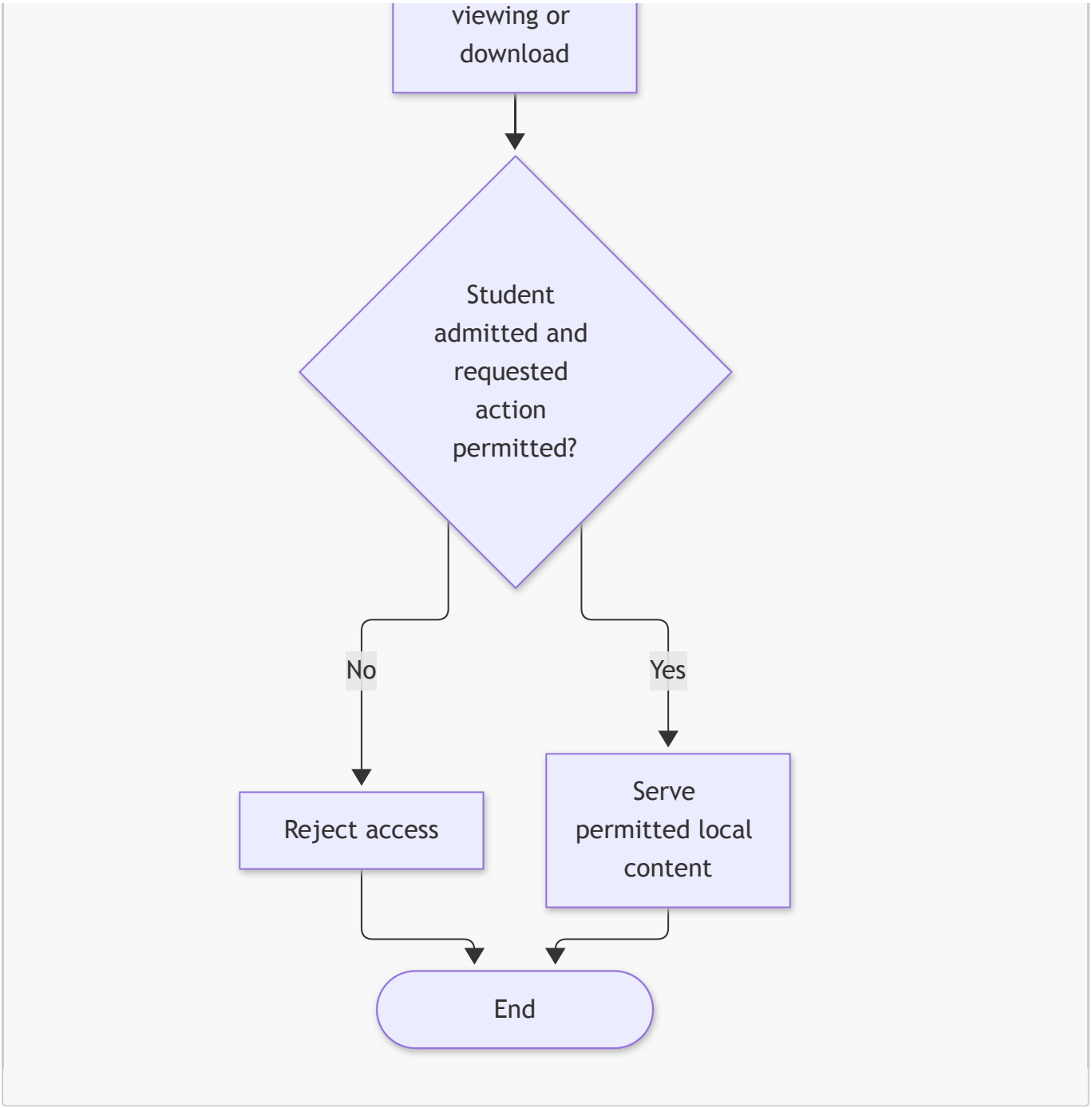


Avoid displaying global “Healthy” when only the web page is reachable. Media can fail while the API works; DNS filtering can fail while media works. Logs must avoid secrets and unnecessary browsing-history collection (NAC17/NAC19).

Flow 16 — Share and download materials — Should

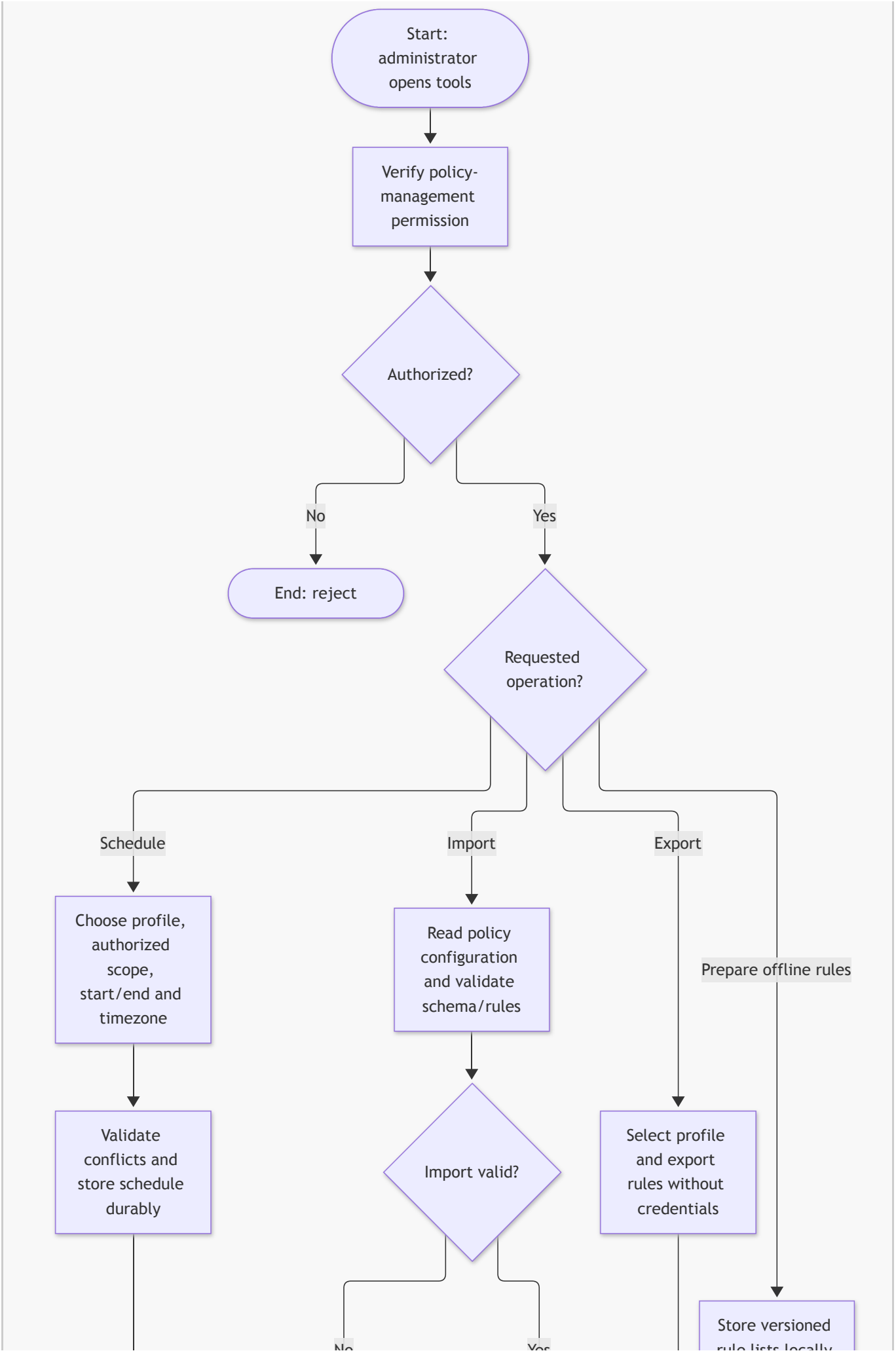
Covers T10 and S09. Displaying a PDF inside a shared screen is already possible through screen capture. Uploading a file and giving students download permission is an additional application workflow. The exact upload/transfer model is a proposed interpretation to confirm.

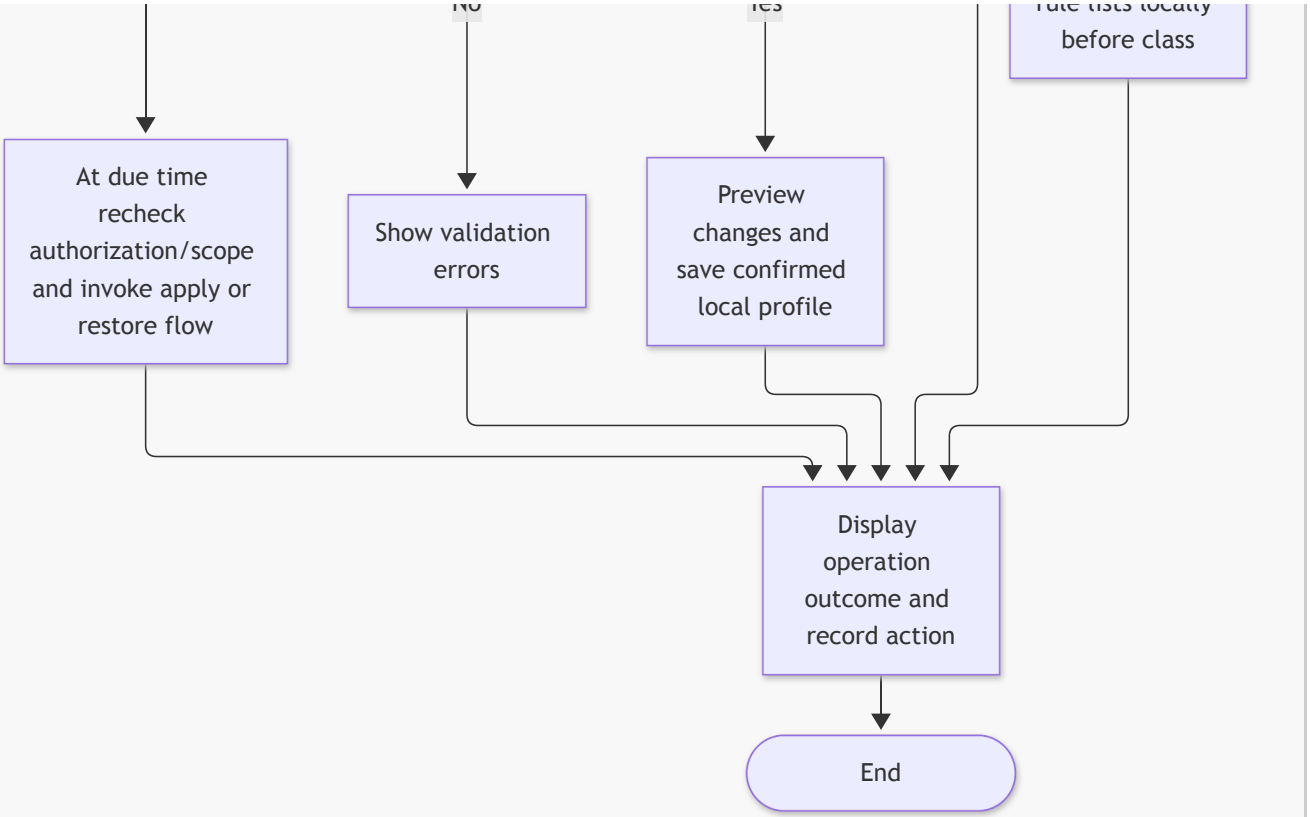

Start
Teacher selects a material file and classroom
Backend checks authority, allowed type and size
Valid?
No
Yes
Show upload/selection error
End
Store material locally and set viewing/download permissions
Show material to admitted students
Student requests



Flow 17 — Policy schedules and configuration transfer — Should

Covers NAC09, NAC14, NAC20. Local rules are also necessary for the broader offline operation requirement; clarify that overlap rather than allowing the global requirement to disappear behind a Should label.

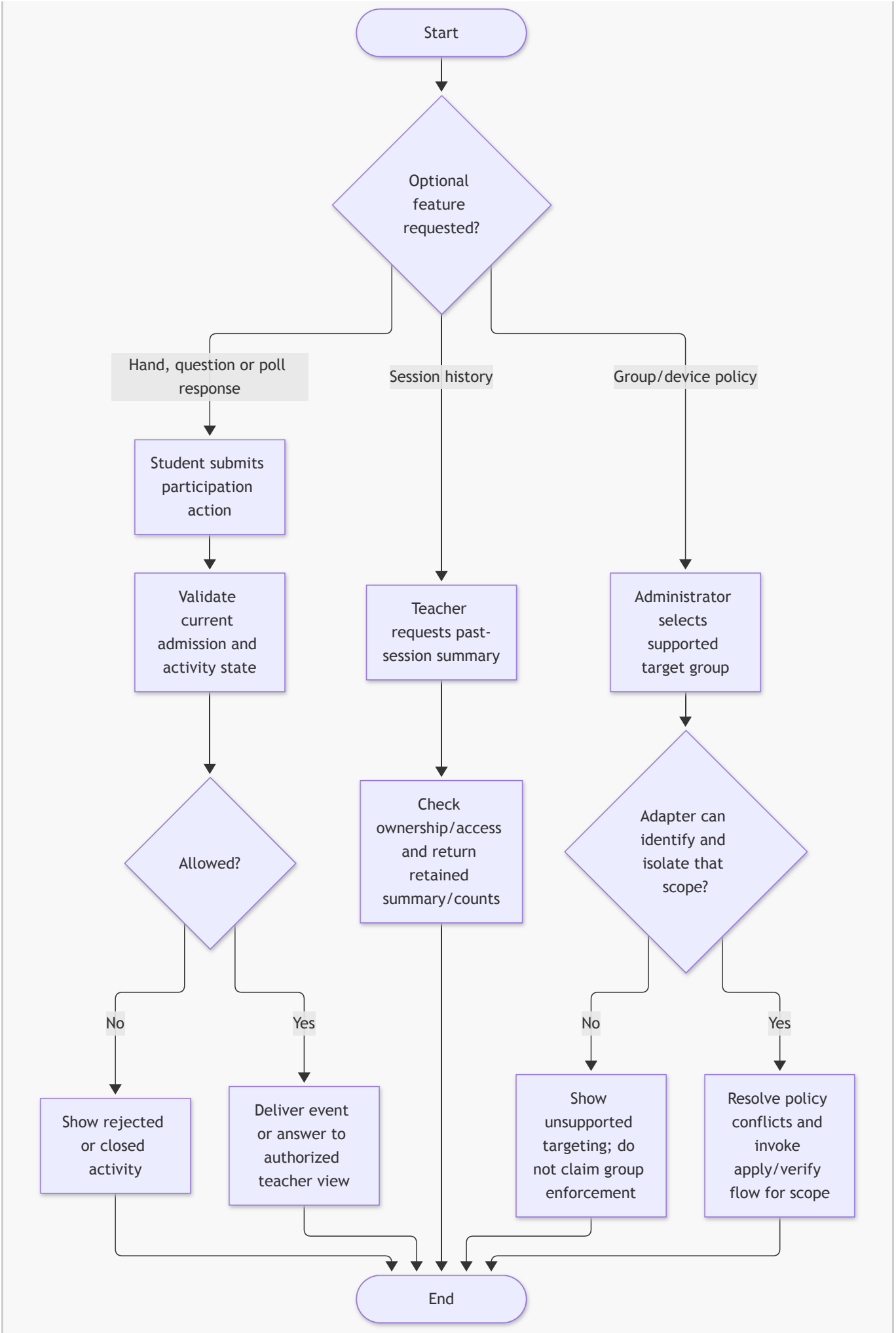

Start: administrator opens tools
Verify policy-management permission
Authorized?
No
Yes
End: reject
Requested operation?
Schedule
Import
Export
Prepare offline rules
Choose profile, authorized scope, start/end and timezone
Read policy configuration and validate schema/rules
Validate conflicts and store schedule durably
Import valid?
Select profile and export rules without credentials
No
Yes
Store versioned



Importing or scheduling a profile does not itself prove enforcement. Scheduled operations must still verify outcomes. Timezone and clock behavior during offline operation need agreement.

Flow 18 — Optional participation, history and group policies — Could

Covers S10, T11, NAC15. These are three distinct optional features, shown as separate branches so they do not become hidden Must work.


Start
Optional feature requested?
Hand, question or poll response
Session history
Group/device policy
Student submits participation action
Validate current admission and activity state
Teacher requests past-session summary
Administrator selects supported target group
Allowed?
Check ownership/access and return retained summary/counts
Adapter can identify and isolate that scope?
No
Yes
No
Yes
Show rejected or closed activity
Deliver event or answer to authorized teacher view
Show unsupported targeting; do not claim group enforcement
Resolve policy conflicts and invoke apply/verify flow for scope
End

For polls, question creation/open/close/results behavior needs refinement; the source lists the feature but does not specify the complete poll product. Session history does not imply recording the teacher's screen. Per-device network identity cannot automatically be inferred from a browser nickname, especially behind shared network translation.

5. Full requirements-to-flow index

Every supplied ID is represented below. A reference means the requirement is explained; it does not mean it is implemented or accepted.

IDs	Priority / classification	Flows
T01, T08	Must	04, 05
T02, T03, T04, T07	Must	06, 07
T05, T06	Must	05, 08
T09	Should	15
T10	Should	16
T11	Could	18
S01, S02	Must	05
S03, S04, S05	Must	06, 07
S06, S07	Must	05, 07, 12, 15
S08	Should	05
S09	Should	16
S10	Could	18
NAC01, NAC05	Must	01, 10
NAC02, NAC03, NAC04, NAC07, NAC08	Must	09, 10, 11
NAC06	Must	13, 14
NAC09	Should	17
NAC10, NAC11	Must	10, 13, 14, 15
NAC12, NAC13	Must	14, 15
NAC14	Should	17
NAC15	Could	18
NAC16, NAC17, NAC18, NAC19	Must / cross-cutting	09, 10, 13, 14, 15

IDs	Priority / classification	Flows
NAC20	Should; global offline overlap	03, 17
CN01, CN02, CN03, CN04, CN05, CN06, CN07, CN08, CN09, CN10	Campus deployment obligations	02
OR01, OR02, OR03, OR04, OR05, OR06, OR07, OR08, OR09, OR10, OR11, OR12	Portable deployment obligations	03

6. What belongs in each software layer?

Layer	Stores or processes	Does not by itself establish
Frontend	Teacher/student/admin interaction and visible status	Actual restriction of other devices
Backend	Identity/authorization, session data, policy lifecycle, verified outcomes and audit	Media distribution or network enforcement without integrations
Database	Users, profiles, sessions, participant/admission records, policy restoration records	A live video transport
Signaling / SFU	Media connection negotiation, forwarding audio/video to authorized participants	Domain/category filtering
Enforcement service / adapter	Rules, supported network scope and read-back/probes	Prevention of all VPN/offline-tool bypass
Local infrastructure	Names, trusted HTTPS, DHCP, reachability, media traffic, resolver path and Wi-Fi capacity	Proven capacity until measured

At 50–150 viewers, the required SFU design keeps the teacher from sending a separate full stream directly to every student. The server and Wi-Fi infrastructure still need enough capacity to forward all viewer traffic. This is why infrastructure testing is part of the project, not an installation detail to postpone until the demo.

7. Decisions to agree before detailed implementation

Decision	Why it changes behavior
Are teacher/admin accounts pre-provisioned, and can one person have both roles?	Determines who can create classes versus edit/apply profiles
Is manual teacher approval required for every join?	Determines whether code entry leads to waiting or immediate viewing

Decision	Why it changes behavior
What identifies a student, and how long does a block last?	Determines realistic removal/rejoin guarantees and collected data
What exactly counts as starting a class?	Defines the event that automatically applies its approved profile
Must an exam stop if restrictions cannot be verified?	Determines the failure branch in Flow 01
Which network devices are affected, and can multiple classes share the scope?	Prevents one class changing another class's browsing policy
What are rule precedence and restricted-by-default semantics for the chosen adapter?	Makes exceptions and filtering tests predictable
What does pause do to video and microphone audio?	Avoids unexpected continued capture/publication
Which browser/OS/audio-source combinations are supported?	Defines feasible capture and acceptance tests
How are HTTPS trust, local names and certificate renewal provisioned?	Determines whether browser capture and connection work offline
What are teacher-disconnect, rule expiry and restore-retry behaviors?	Prevents restrictions persisting indefinitely or ending unexpectedly
What physical capacity and performance measurements prove acceptance?	Separates tested 50/150-device behavior from target claims
What logs/identifiers are retained, for how long, and who can access them?	Implements minimal browsing-history collection and operational support

These are clarification points, not new mandatory features. Resolve them through examples and small technical tests, then use the agreed flows to divide implementation work and plan sprints.